Attention Is All You Need ("Transformers")

Michael Schatz
November 9, 2026
Applied Comparative Genomics

Project Prelim Report
Due: Wednesday Nov 11 by 11:59pm

Preliminary Project Report

Assignment Date: Monday November 2, 2026
Due Date: Wednesday, November 11, 2026 @ 11:59pm

Each team should submit a PDF of your preliminary project proposal (2 to 3 pages) to GradeScope by 11:59pm on Wednesday November 11.

The preliminary report should have at least:

- Title of your project
- List of team members and email addresses
- 1 paragraph abstract summarizing the project
- 1+ paragraph of Introduction
- 1+ paragraph of Methods that you are using
- 1+ paragraph of Results, describing the data evaluated and any preliminary results
- 1+ paragraph of Discussion (what you have seen or expect to see)
- 1+ figure showing a preliminary result (typically a summary of the data you have identified for your project)
- 5+ References to relevant papers and data

The preliminary report must use the Bioinformatics style template. Word and LaTeX templates are available in the project directory. Overleaf is recommended for LaTex submissions. Google Docs is recommended for non-latex submissions, especially group projects. Paperpile is recommended for citation management.

Later, you will present your project in class starting the week of December 2. You will also submit your final written report (6-8 pages) of your project by Dec 17 with a draft due on Dec 13 and a final project presentation on Dec 14-17

Please use Piazza if you have any questions!

https://github.com/schatzlab/appliedgenomics2026/blob/main/project/prelimreport.md

Assignment 5
Due: ***Sunday*** Nov 15 by 11:59pm

Assignment 5: Convolutional Neural Networks and Transformers

Assignment Date: Monday, November 9, 2026
Due Date: Sunday, November 15 @ 11:59pm

Assignment Overview

In this assignment you will explore a couple of key aspects of convolutional neural networks such as self-attention and feature encoding as well as explore a pre-trained convolutional neural network for gene expression prediction. For this assignment, we will provide a Jupyter notebook with code for you to use and complete your assignment in.

For this assignment, you will create the environment as follows:

```
mamba create -n asn5 python=3.10 scikit-learn pytorch matplotlib pandas numpy jupyter seaborn kipoiseq logomaker
```

If installation fails due to "warning: libmamba - Problem type not implemented SOLVER_RULE_STRICT_REPO_PRIORITY", try running the following and then attempt the above command again:

```
conda config --set channel_priority flexible
```

Then activate the environment and install the following package using pip:

```
mamba activate asn5
```

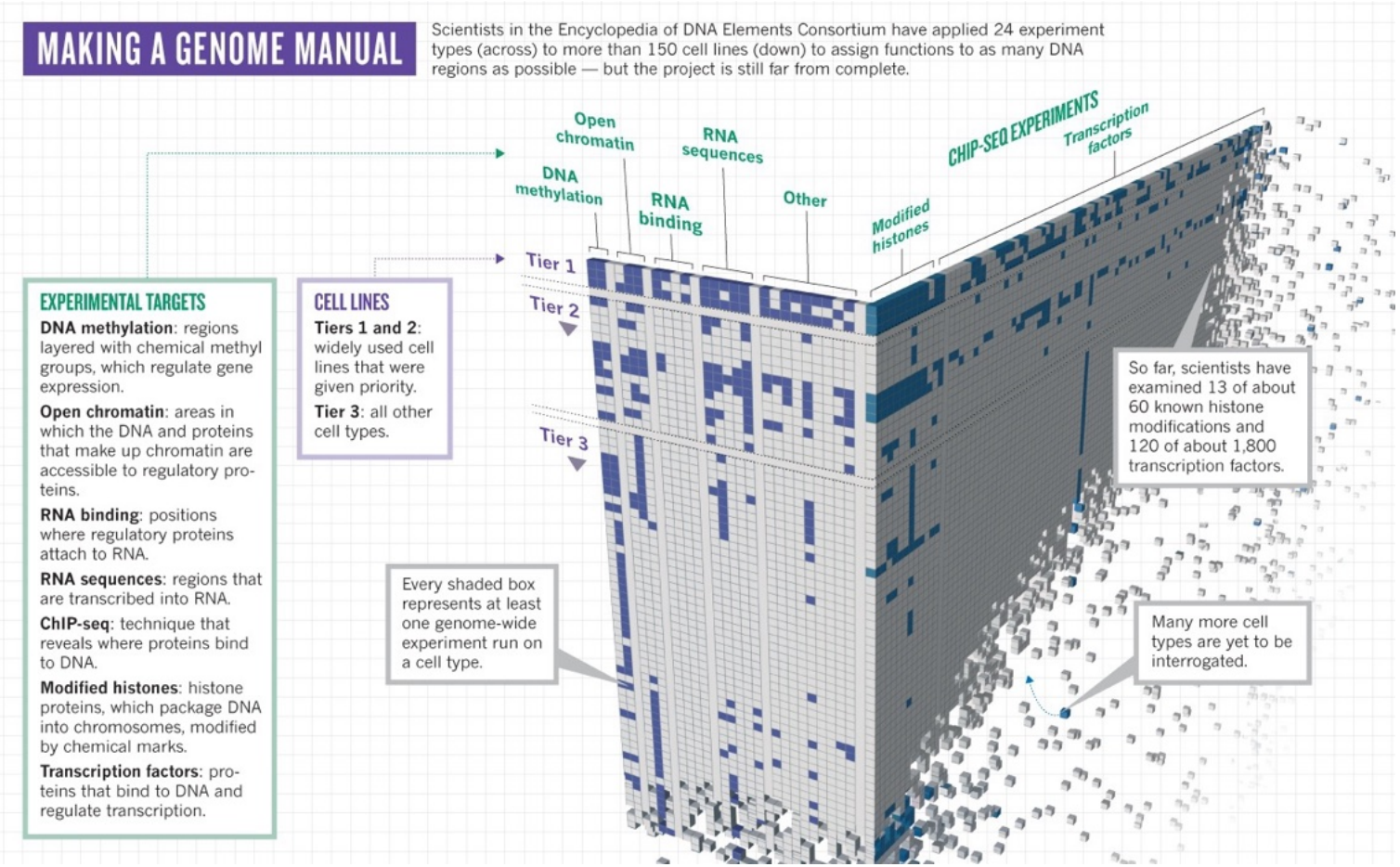
```
pip install enformer-pytorch
```

If you have issues with creating the environment and installing the required packages, you can use Google Colab

https://github.com/schatzlab/appliedgenomics2026/tree/main/assignments/assignment5

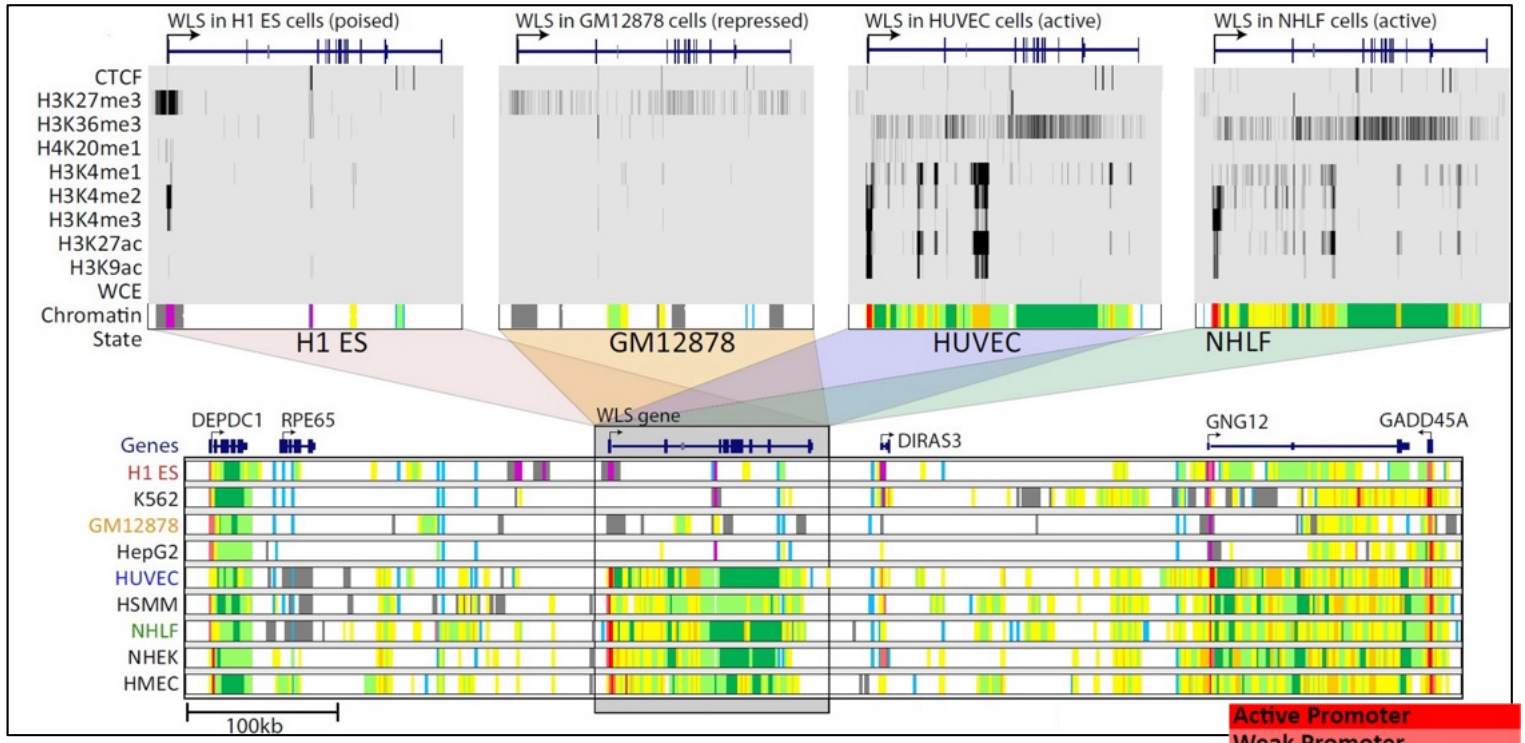
Check Piazza for questions!

ENCODE Data Sets



1,640 data sets total over 147 different cell types

Chromatin states dynamics across nine cell types



- **Single annotation track for each cell type**
- **Summarize cell-type activity at a glance**
- **Can study 9-cell activity pattern across ↓**

Ernst et al, Nature 2011

Attention Is All You Need ("Transformers")

“Transformers”

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.



Ashish Vaswani
Essential AI



Llion Jones
Sakana AI



Noam Shazeer
Character AI



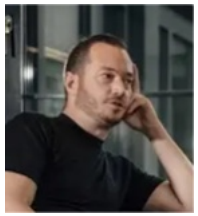
Aidan Gomez
Cohere



Niki Parmar
Essential AI



Łukasz Kaiser
OpenAI

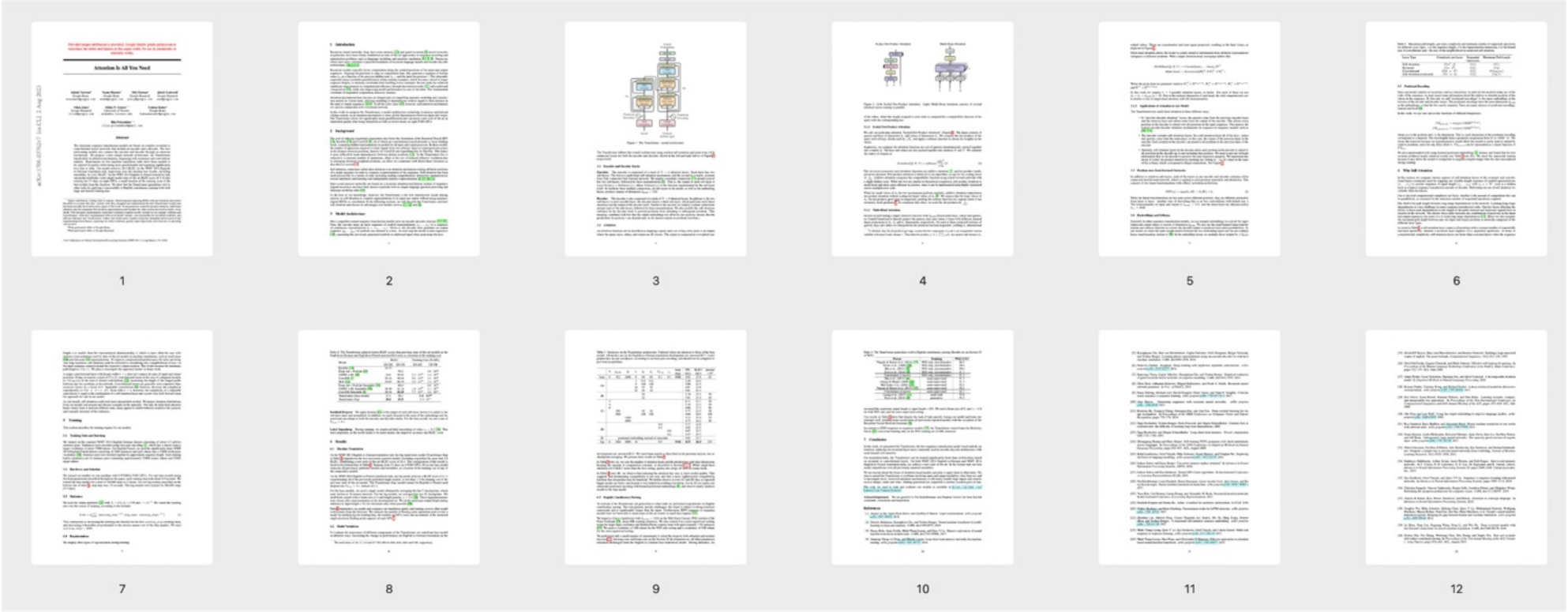


Jakob Uszkoreit
Inceptive



Illia Polosukhin
NEAR

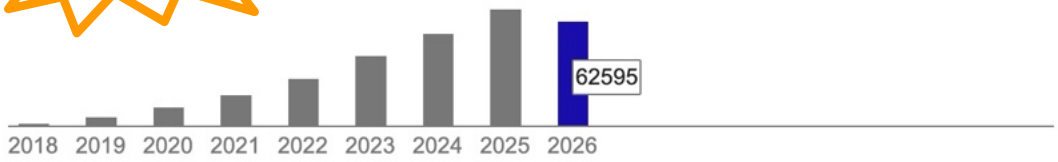
The paper



Vaswani et al (2017) arXiv:1706.03762

Attention is all you need

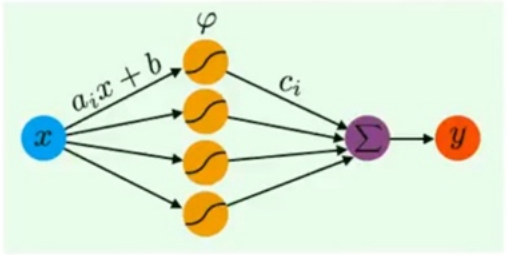
Authors	Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, Illia Polosukhin
Publication date	2017
Journal	Advances in neural information processing systems
Volume	30
Description	The dominant sequence transduction models are based on complex recurrent orconvolutional neural networks in an encoder and decoder configuration. The best performing such models also connect the encoder and decoder through an attentionm echanisms. We propose a novel, simple network architecture based solely onan attention mechanism, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superiorin quality while being more parallelizable and requiring significantly less timeto train. Our single model with 165 million parameters, achieves 27.5 BLEU onEnglish-to-German translation, improving over the existing best ensemble result by over 1 BLEU. On English-to-French translation, we outperform the previoussingle state-of-the-art with model by 0.7 BLEU, achieving a BLEU score of 41.1.
Total citations	Cited by 296563



Outline

1. Recap on ANNs and CNNs
2. The problem
3. Self-Attention
4. Transformers
5. Impact

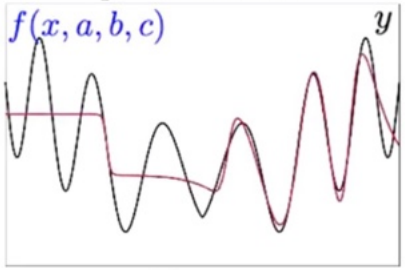
ANNs are “Universal Approximators”



1 hidden layer perceptron:

$$y \approx f(x, a, b, c) \stackrel{\text{def.}}{=} \sum_{i=1}^{p} c_i \varphi(a_i x + b_i)$$

$p = 6$ neurons



$p = 20$ neurons



Approximation by Superpositions of a Sigmoidal Function*

G. Cybenko†

Abstract. In this paper we demonstrate that finite linear combinations of compositions of a fixed, univariate function and a set of affine functionals can uniformly approximate any continuous function of n real variables with support in the unit hypercube; only mild conditions are imposed on the univariate function. Our results settle an open question about representability in the class of single hidden layer neural networks. In particular, we show that arbitrary decision regions can be arbitrarily well approximated by continuous feedforward neural networks with only a single internal, hidden layer and any continuous sigmoidal nonlinearity. The paper discusses approximation properties of other possible types of nonlinearities that might be implemented by artificial neural networks.

Key words. Neural networks, Approximation, Completeness.



ConvnetJS demo: toy 2d classification with 2-layer neural network

The simulation below shows a toy binary problem with a few data points of class 0 (red) and 1 (green). The network is set up as:

```
layer_defs = [];
layer_defs.push({type:'input', out_sx:1, out_sy:1, out_depth:2});
layer_defs.push({type:'fc', num_neurons:6, activation: 'tanh'});
layer_defs.push({type:'fc', num_neurons:2, activation: 'tanh'});
layer_defs.push({type:'softmax', num_classes:2});

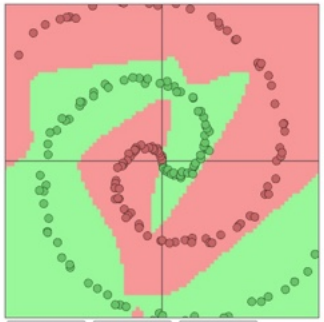
net = new convnetjs.Net();
net.makeLayers(layer_defs);

trainer = new convnetjs.SGDTrainer(net, {learning_rate:0.01, momentum:0.1, batch_size:10, l2_decay:0.001});
```

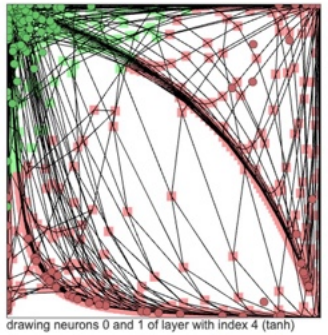
change network

Feel free to change this, the text area above gets eval()'d when you hit the button and the network gets reloaded. Every 10th of a second, all points are fed to the network multiple times through the trainer class to train the network. The resulting predictions of the network are then "painted" under the data points to show you the generalization.

On the right we visualize the transformed representation of all grid points in the original space and the data, for a given layer and only for 2 neurons at a time. The number in the bracket shows the total number of neurons at that level of representation. If the number is more than 2, you will only see the two visualized but you can cycle through all of them with the cycle button.



simple data | circle data | spiral data | random data



drawing neurons 0 and 1 of layer with index 4 (tanh)

fc(6) | tanh(6) | fc(2) | tanh(2) | fc(2)

cycle through visualized neurons at selected layer (if more than 2)

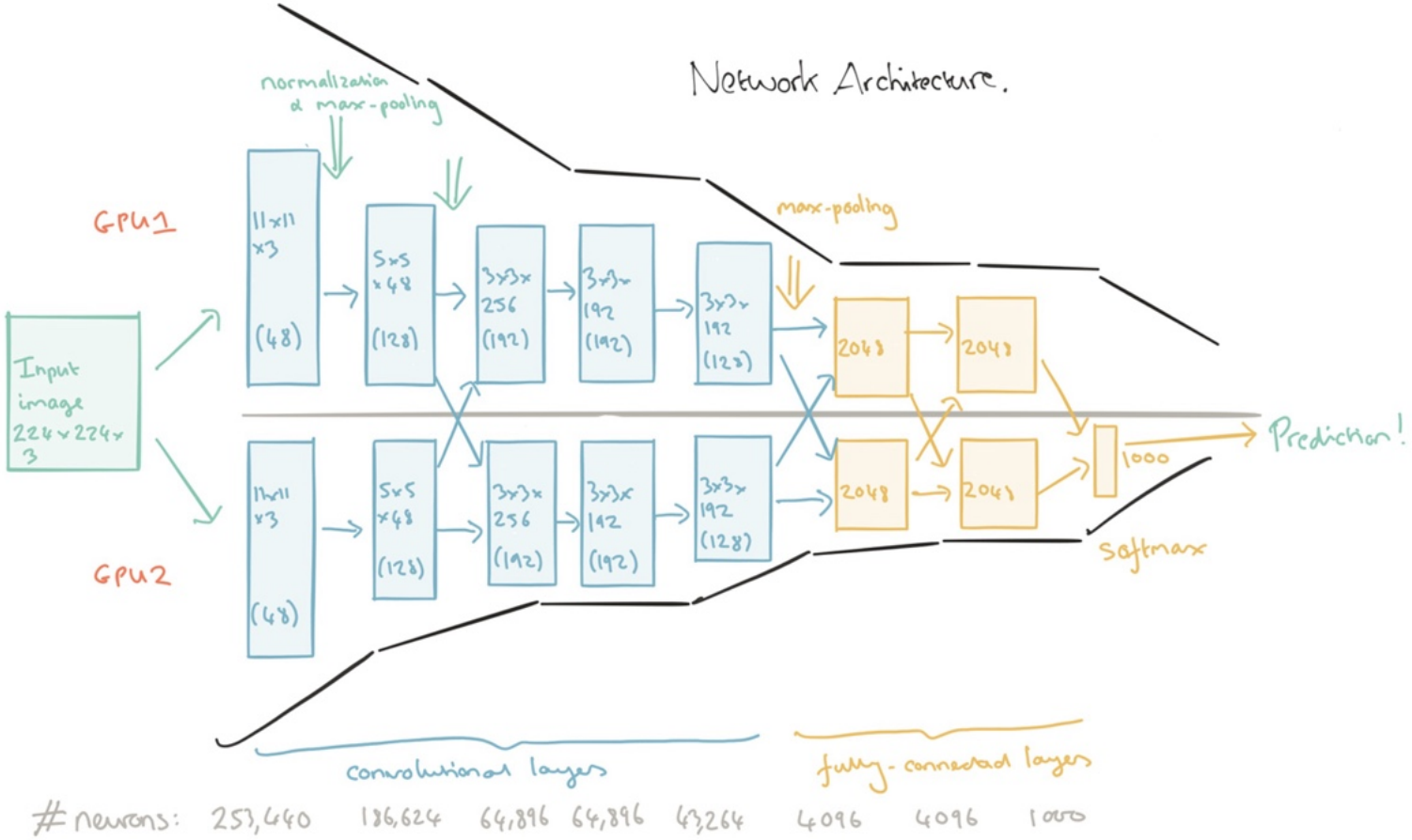
Controls:
CLICK: Add red data point
SHIFT+CLICK: Add green data point
CTRL+CLICK: Remove closest data point

AlexNet Architecture (annotated)









DOG

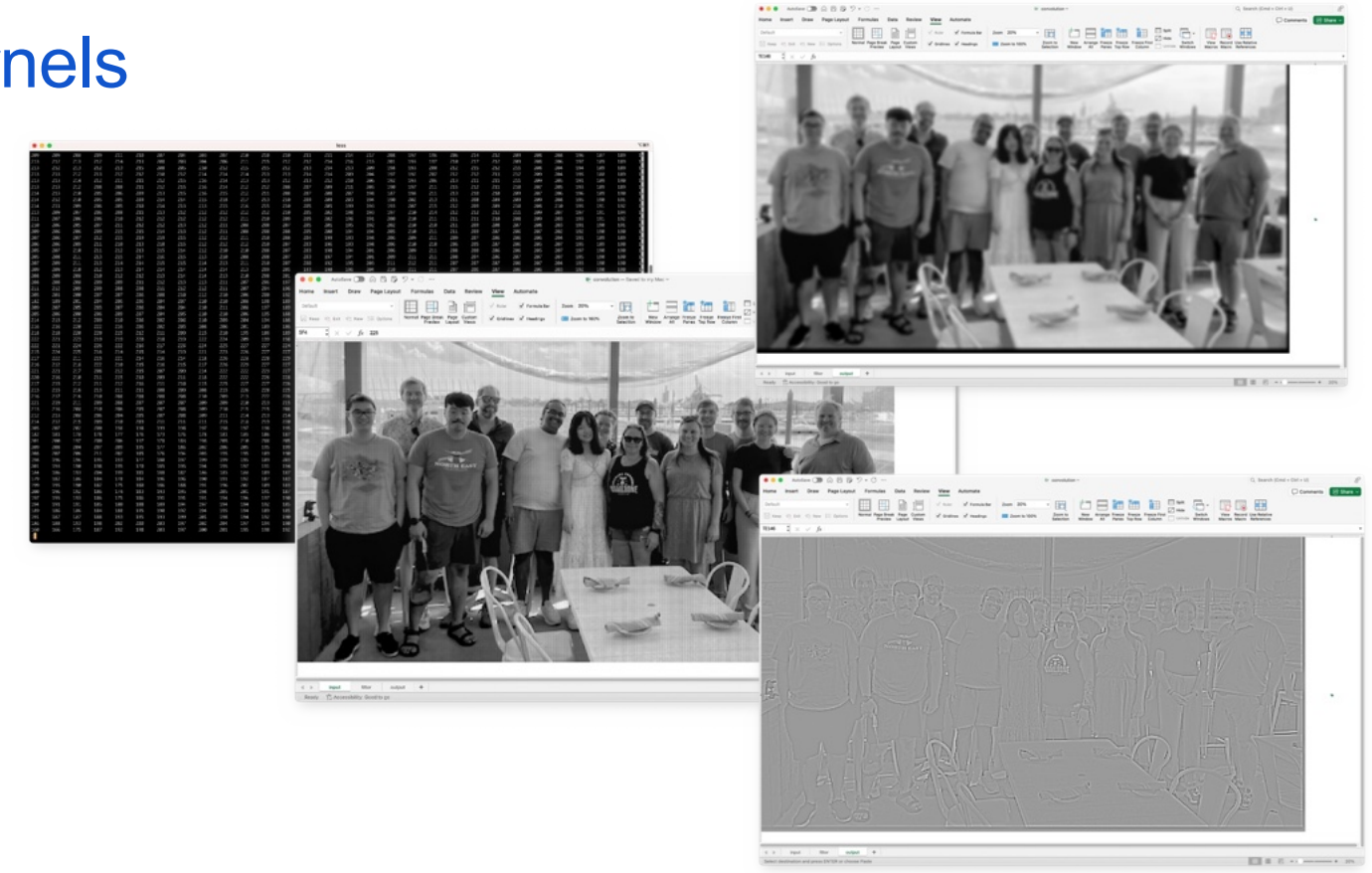
BAGEL

VAN

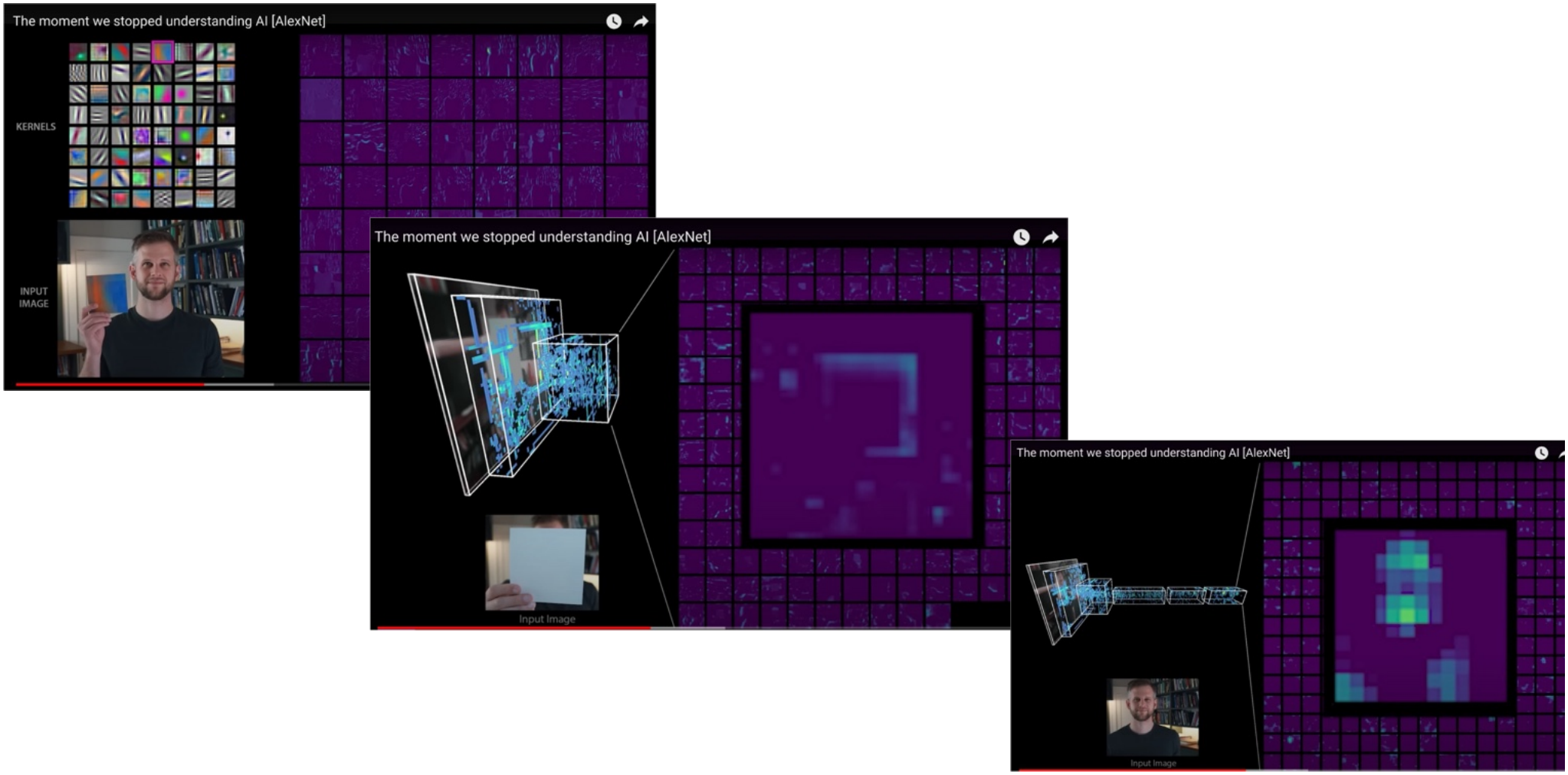
https://blog.acolyer.org/2016/04/20/imagenet-classification-with-deep-convolutional-neural-networks/

Convolutional Kernels

Operation	Kernel ω	Image result g(x,y)
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur 3 × 3 (approximation)	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	
Gaussian blur 5 × 5 (approximation)	$\frac{1}{256}\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	
Unsharp masking 5 × 5 Based on Gaussian blur with amount as 1 and threshold as 0 (with no image mask)	$\frac{-1}{256}\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & -476 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	



= input!A1 * filter!$A$1 + input!A2 * filter!$A$2 + input!A3 * filter!$A$3 + input!A4 * filter!$A$4 + input!A5 * filter!$A$5 +
input!B1 * filter!$B$1 + input!B2 * filter!$B$2 + input!B3 * filter!$B$3 + input!B4 * filter!$B$4 + input!B5 * filter!$B$5 +
input!C1 * filter!$C$1 + input!C2 * filter!$C$2 + input!C3 * filter!$C$3 + input!C4 * filter!$C$4 + input!C5 * filter!$C$5 +
input!D1 * filter!$D$1 + input!D2 * filter!$D$2 + input!D3 * filter!$D$3 + input!D4 * filter!$D$4 + input!D5 * filter!$D$5 +
input!E1 * filter!$E$1 + input!E2 * filter!$E$2 + input!E3 * filter!$E$3 + input!E4 * filter!$E$4 + input!E5 * filter!$E$5

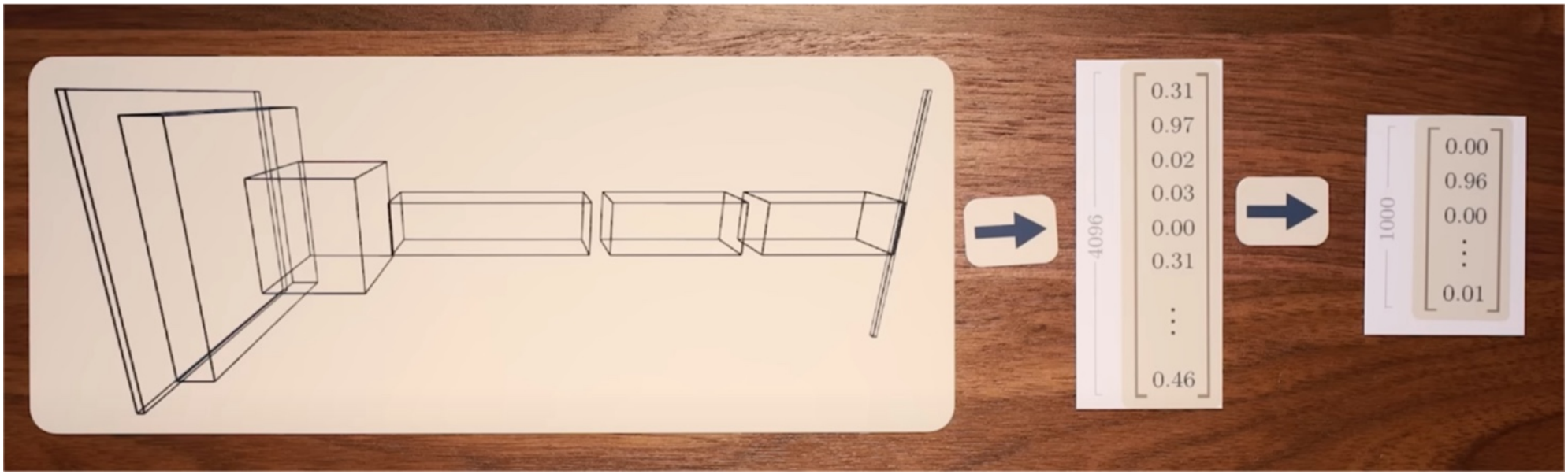


https://www.youtube.com/watch?v=UZDiGooFs54

Final two layers are special:

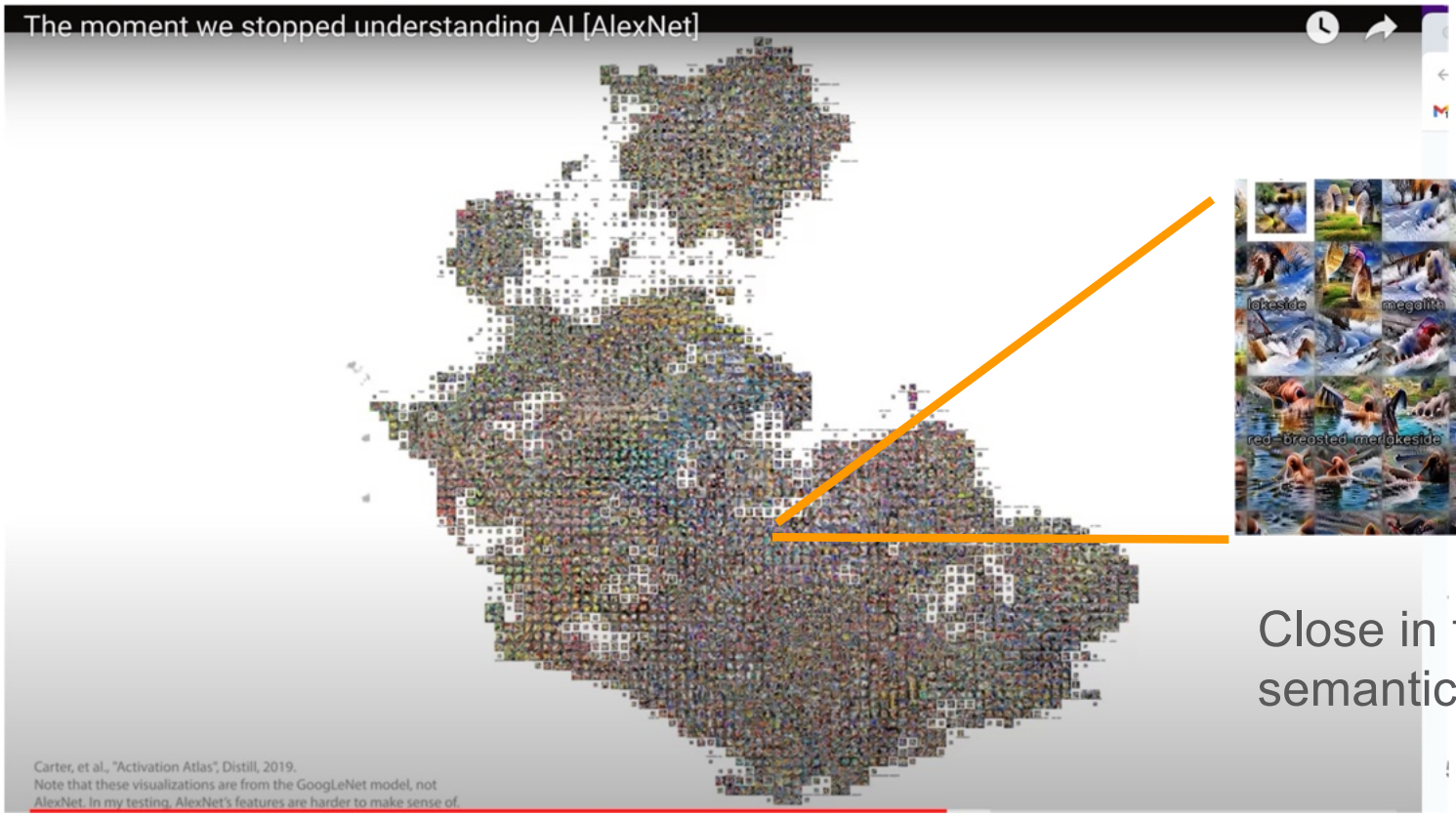
Last layer [1x1000]: Probability of the image in class i

Second last [1x4096]: Projection of image into 4096-dim space



https://www.youtube.com/watch?v=UZDiGooFs54

Activation Atlas: tSNE plot of images embeddings





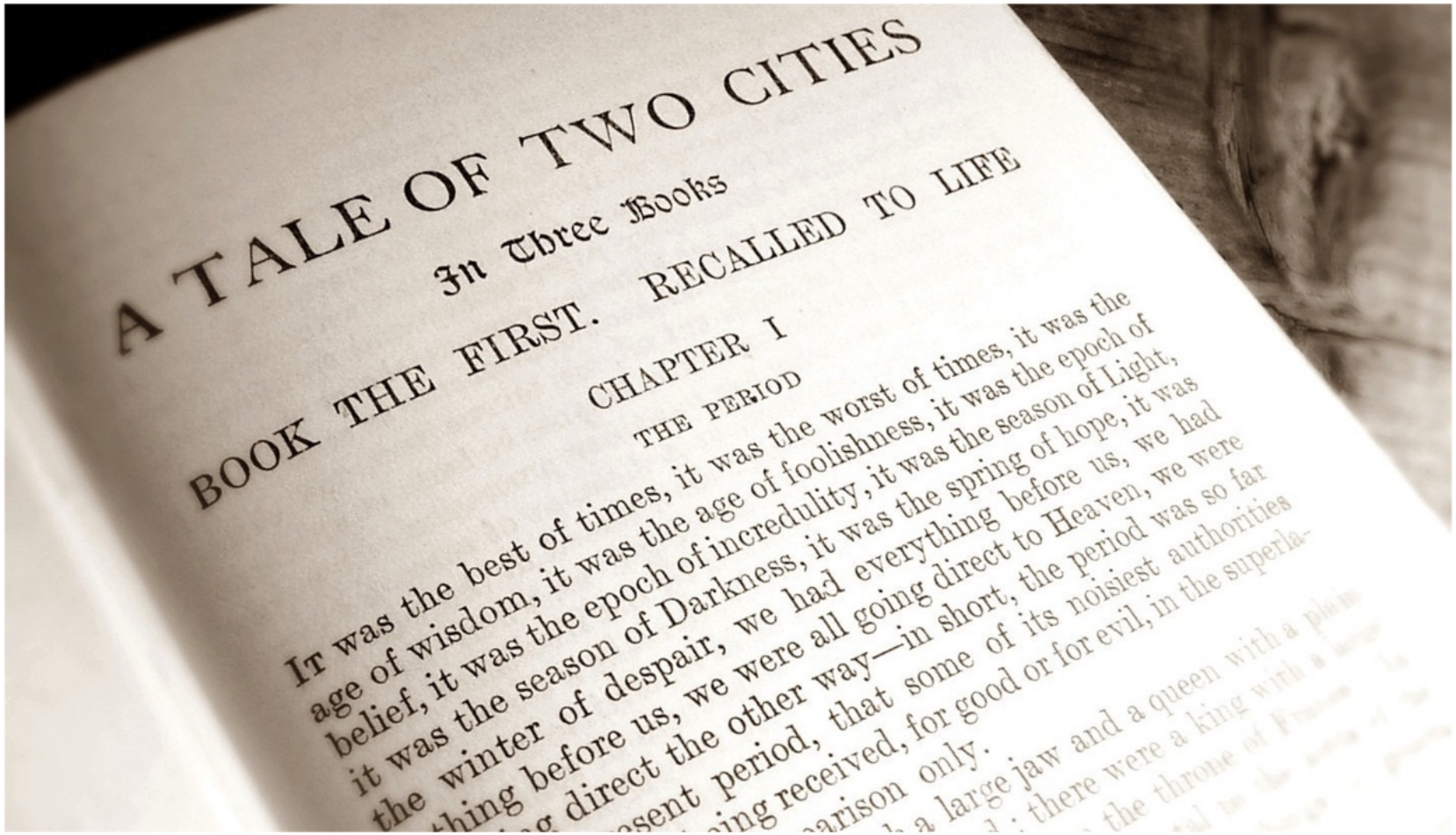
Close in the projection are semantically related

https://www.youtube.com/watch?v=UZDiGooFs54

Outline

1. Recap on ANNs and CNNs
2. The problem
3. Self-Attention
4. Transformers
5. Impact

What about text data?



Or other sequential data?

http://introtodeeplearning.com/



A Sequence Modeling Problem: Predict the Next Word

"This morning I took my cat for a ???

given these words

predict the next word

MIT 6.S191

A Sequence Modeling Problem: Predict the Next Word

"This morning I took my cat for a ???

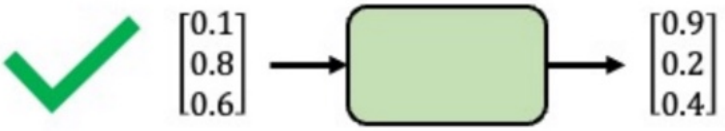
given these words

predict the next word

Representing Language to a Neural Network



Neural networks cannot interpret words



Neural networks require numerical inputs

A Sequence Modeling Problem: Predict the Next Word

f("This morning I took my cat for a")=???

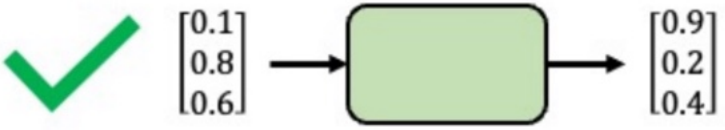
given these words

predict the next word

Representing Language to a Neural Network



Neural networks cannot interpret words



Neural networks require numerical inputs

A Sequence Modeling Problem: Predict the Next Word

"This morning I took my cat for a walk."

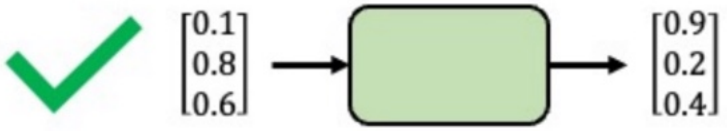
given these words

predict the next word

Representing Language to a Neural Network



Neural networks cannot interpret words

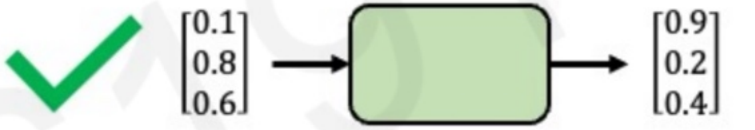


Neural networks require numerical inputs

Encoding Language for a Neural Network

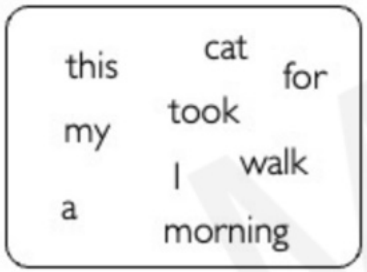


Neural networks cannot interpret words

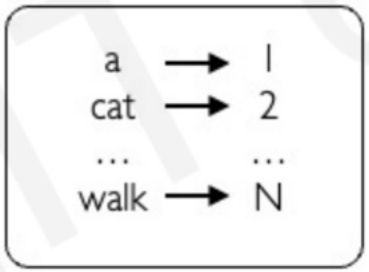


Neural networks require numerical inputs

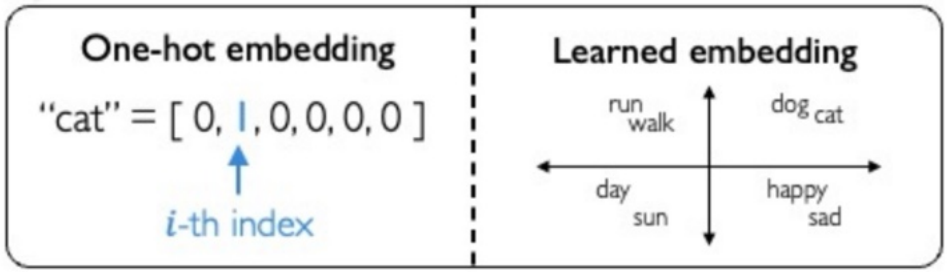
Embedding: transform indexes into a vector of fixed size.



1. Vocabulary:
Corpus of words



2. Indexing:
Word to index



3. Embedding:
Index to fixed-sized vector

A Sequence Modeling Problem: Predict the Next Word

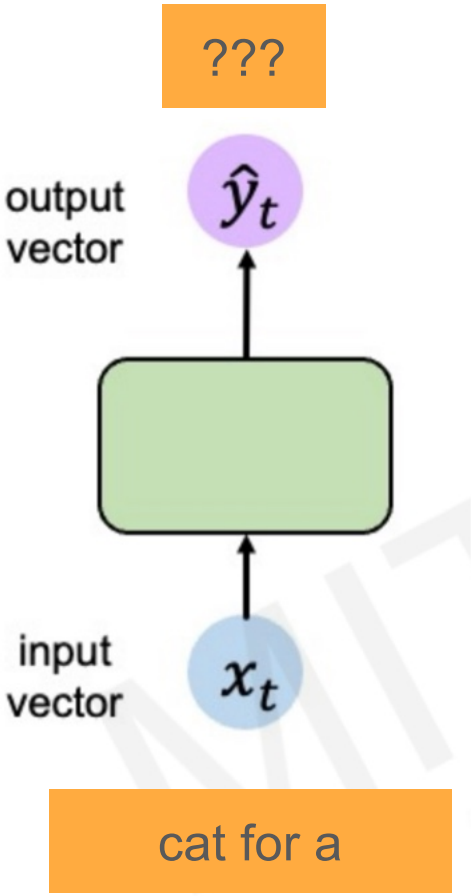
"This morning I took my cat for a ???

given these words

predict the next word

To model sequences, we need to:

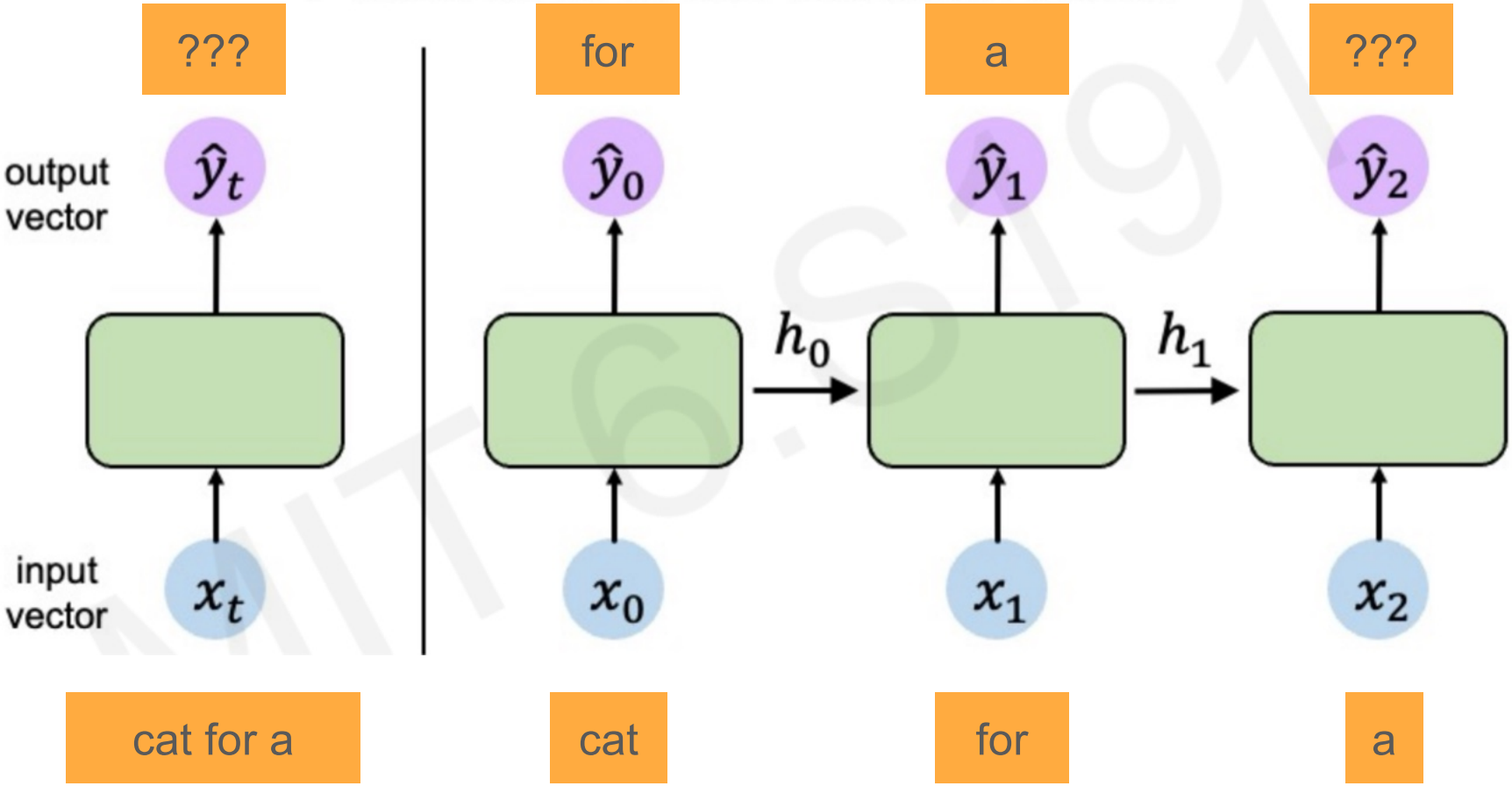
1. Handle **variable-length** sequences
2. Track **long-term** dependencies
3. Maintain information about **order**
4. **Share parameters** across the sequence



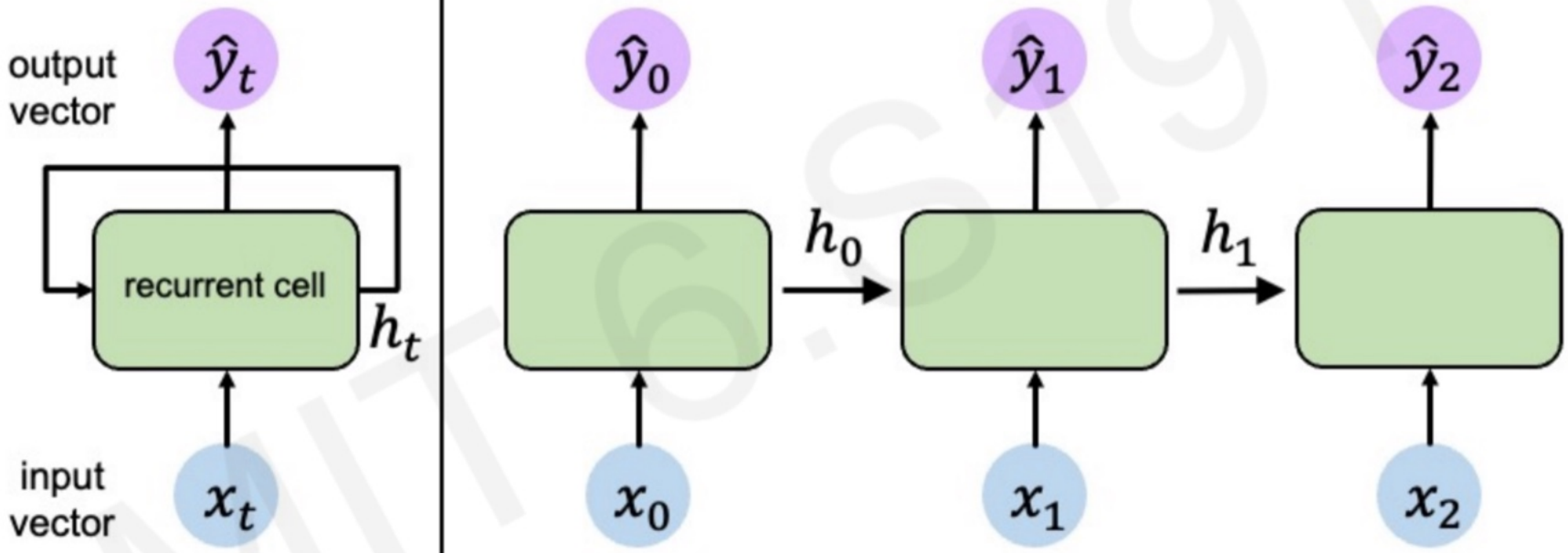
Fixed context networks

- Simple but very limited accuracy
- Quickly runs out of training data for longer windows

Neurons with Recurrence



Neurons with Recurrence

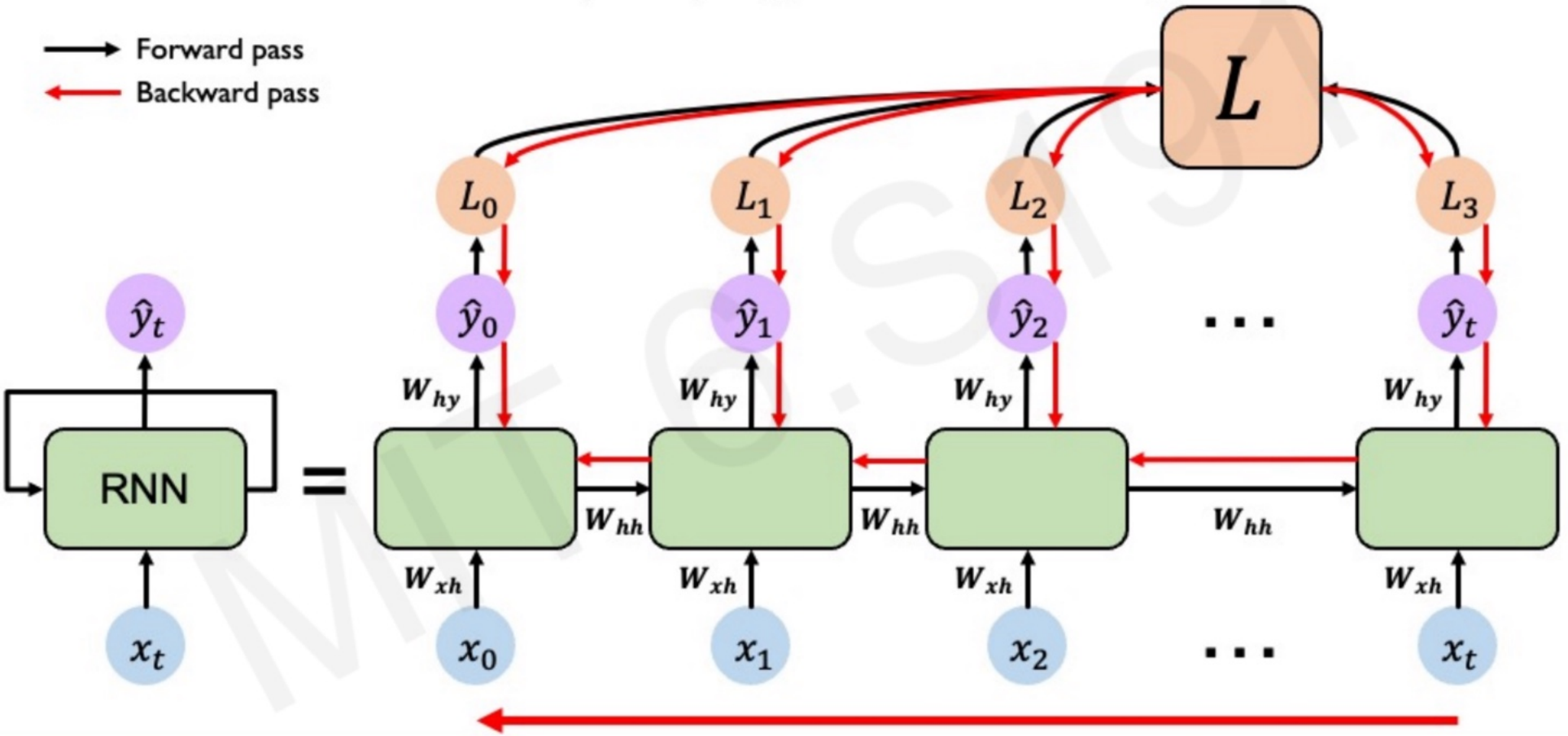


$$\hat{y}_t = f(x_t, h_{t-1})$$

output input past memory

MIT 6.S191

RNNs: Backpropagation Through Time



The Problem of Long-Term Dependencies

Why are vanishing gradients a problem?

Multiply many **small numbers** together

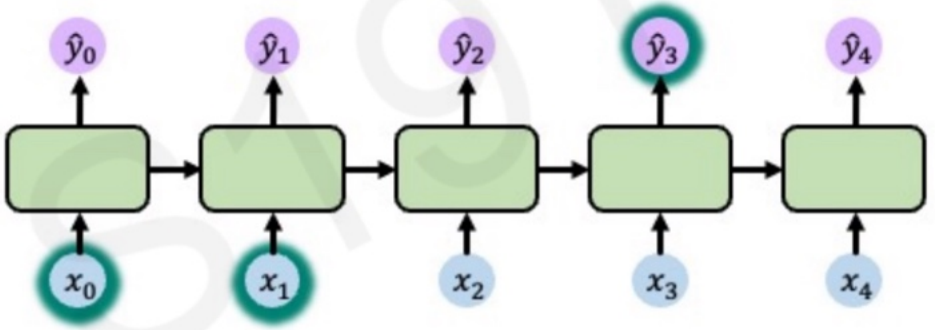
↓

Errors due to further back time steps have smaller and smaller gradients

↓

Bias parameters to capture short-term dependencies

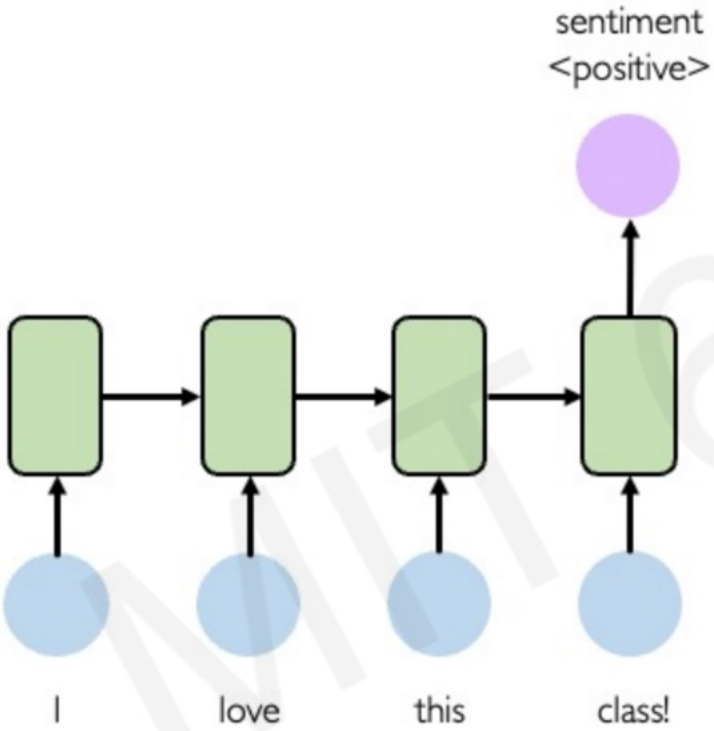
"The clouds are in the ___"



"I grew up in France, … and I speak fluent___ "

MIT 6.S191

Limitations of Recurrent Models

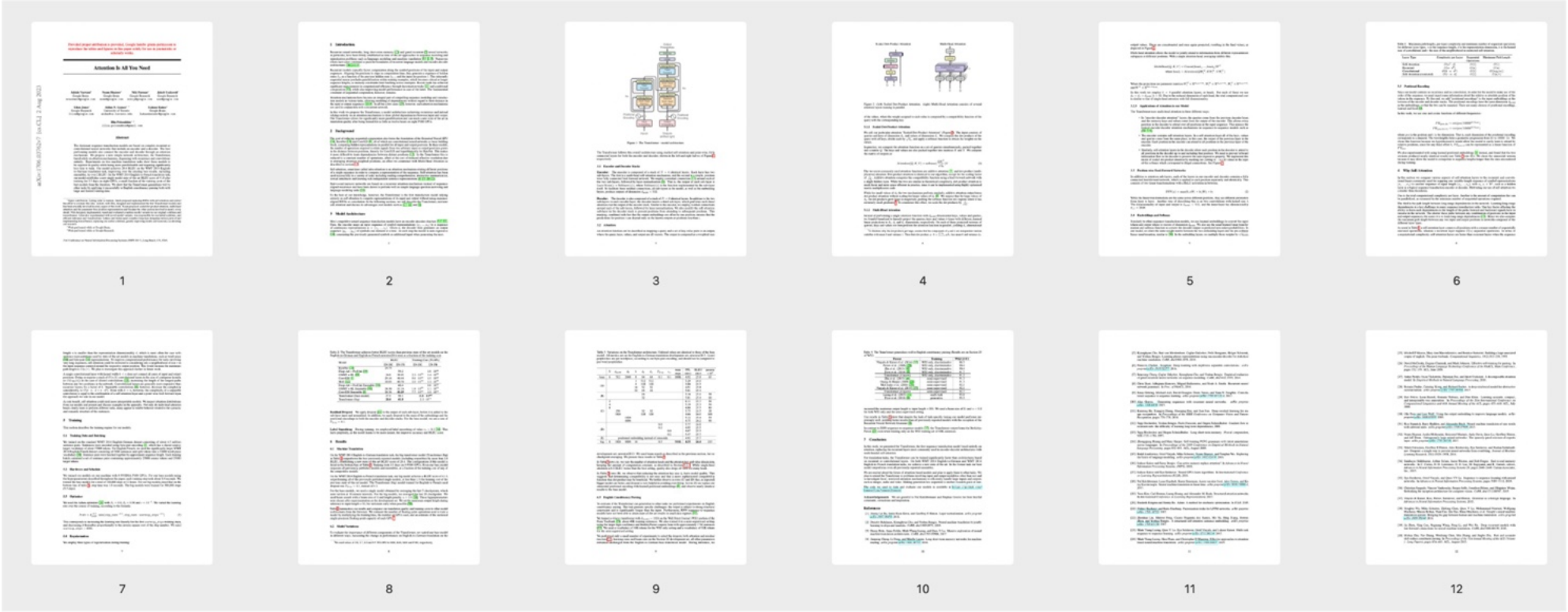


Limitations of RNNs

- Encoding bottleneck
- Slow, no parallelization
- Not long memory

MIT 6.S191

The paper



Vaswani et al (2017) arXiv:1706.03762

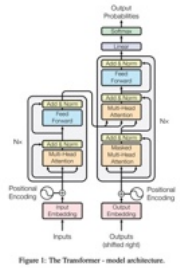
The application

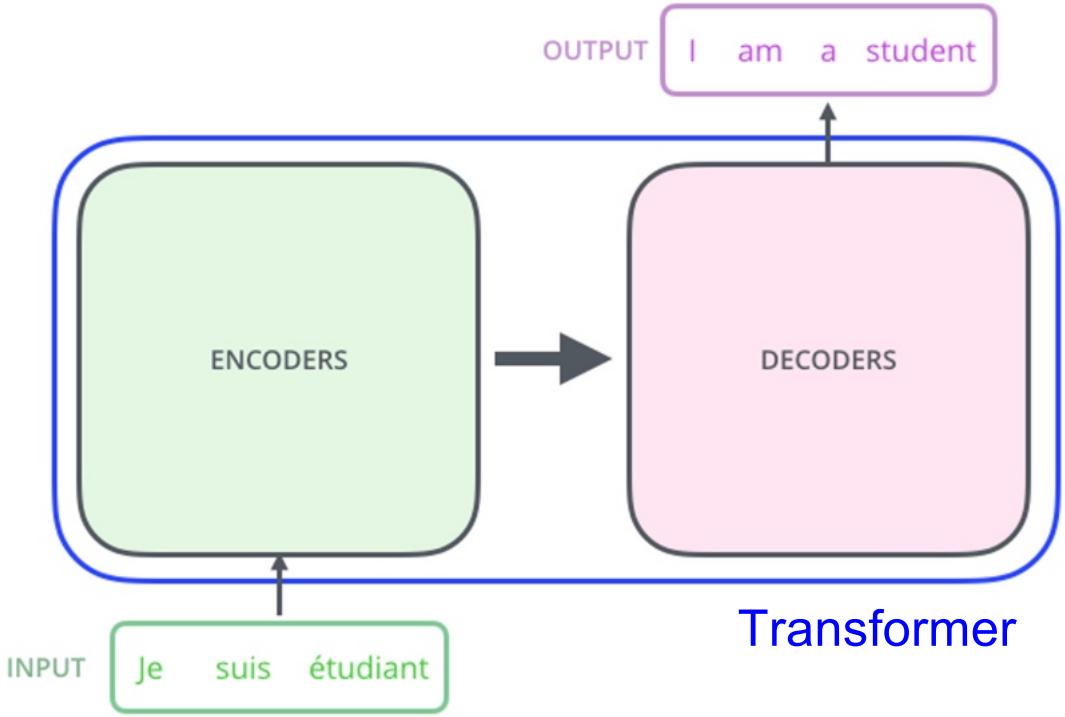


Trained with a huge collection of pairs of sentences in french & english
Measure accuracy using “BLEU” scores to gold standard (higher is better)
Use a “reasonable” amount of GPU time for training (<1 week on 8 GPUs)

https://jalammar.github.io/illustrated-transformer/

The architecture

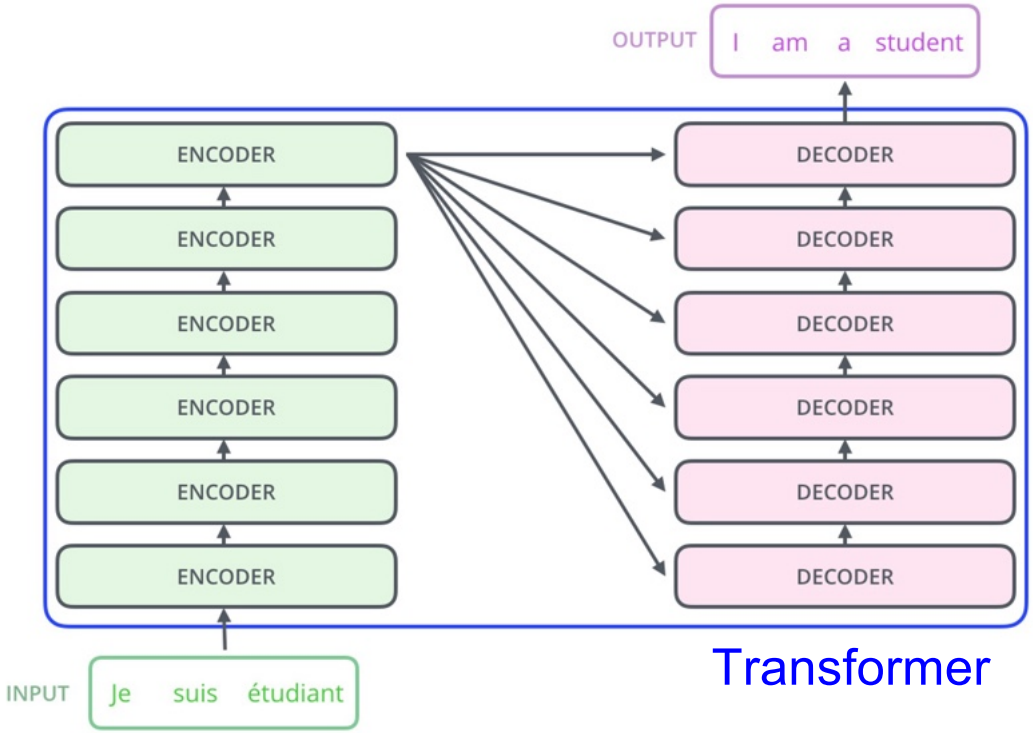




Transformer is composed of an input encoder followed by a output decoder

https://jalammar.github.io/illustrated-transformer/

The architecture



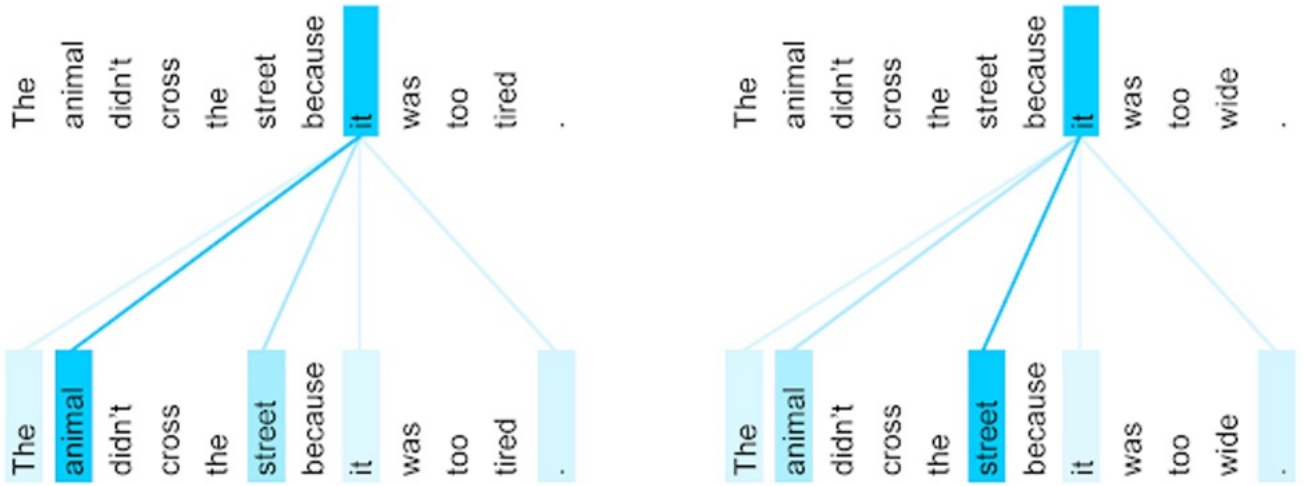
Encoder transforms sentences in language A into a highly abstract embedded space
Decoders progressively decodes highly abstract embedded space into language B

https://jalammar.github.io/illustrated-transformer/

Coreference resolution: self attention

The animal didn't cross the street because it was too tired.
L'animal n'a pas traversé la rue parce qu'il était trop fatigué.

The animal didn't cross the street because it was too wide.
L'animal n'a pas traversé la rue parce qu'elle était trop large.



https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/

Outline

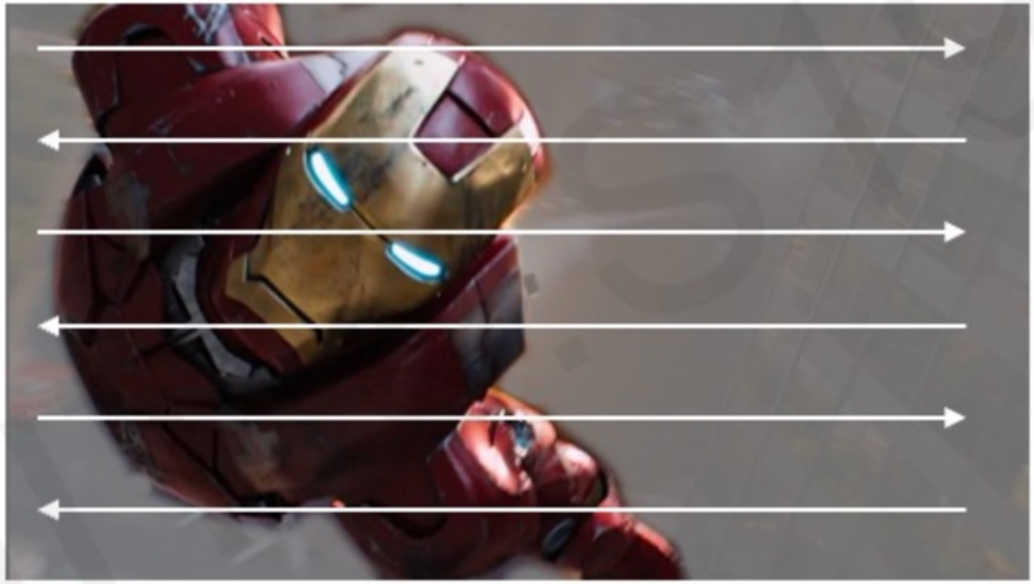
1. Recap on ANNs and CNNs
2. The problem
3. Self-Attention
4. Transformers
5. Impact



What’s important about this image? Where should my attention go?

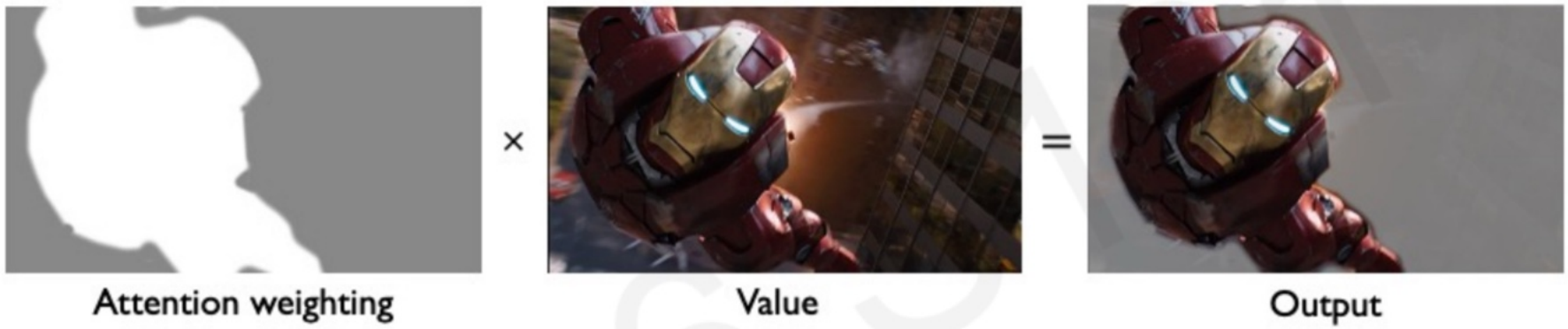
Intuition Behind Self-Attention

Attending to the most important parts of an input.



1. Identify which parts to attend to
2. Extract the features with high attention

Similar to a search problem!



Attention weighting = Query (ironman) * Keys (ironman * learned feature masks)

Note this is a self-attention since the query, keys, and value are the same input just the masks (weights) are different

Applying Multiple Self-Attention Heads



Attention weighting

×



Value

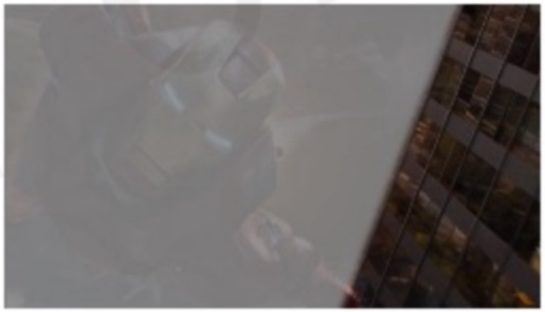
=



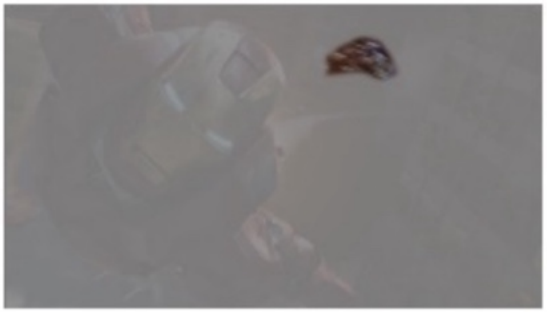
Output



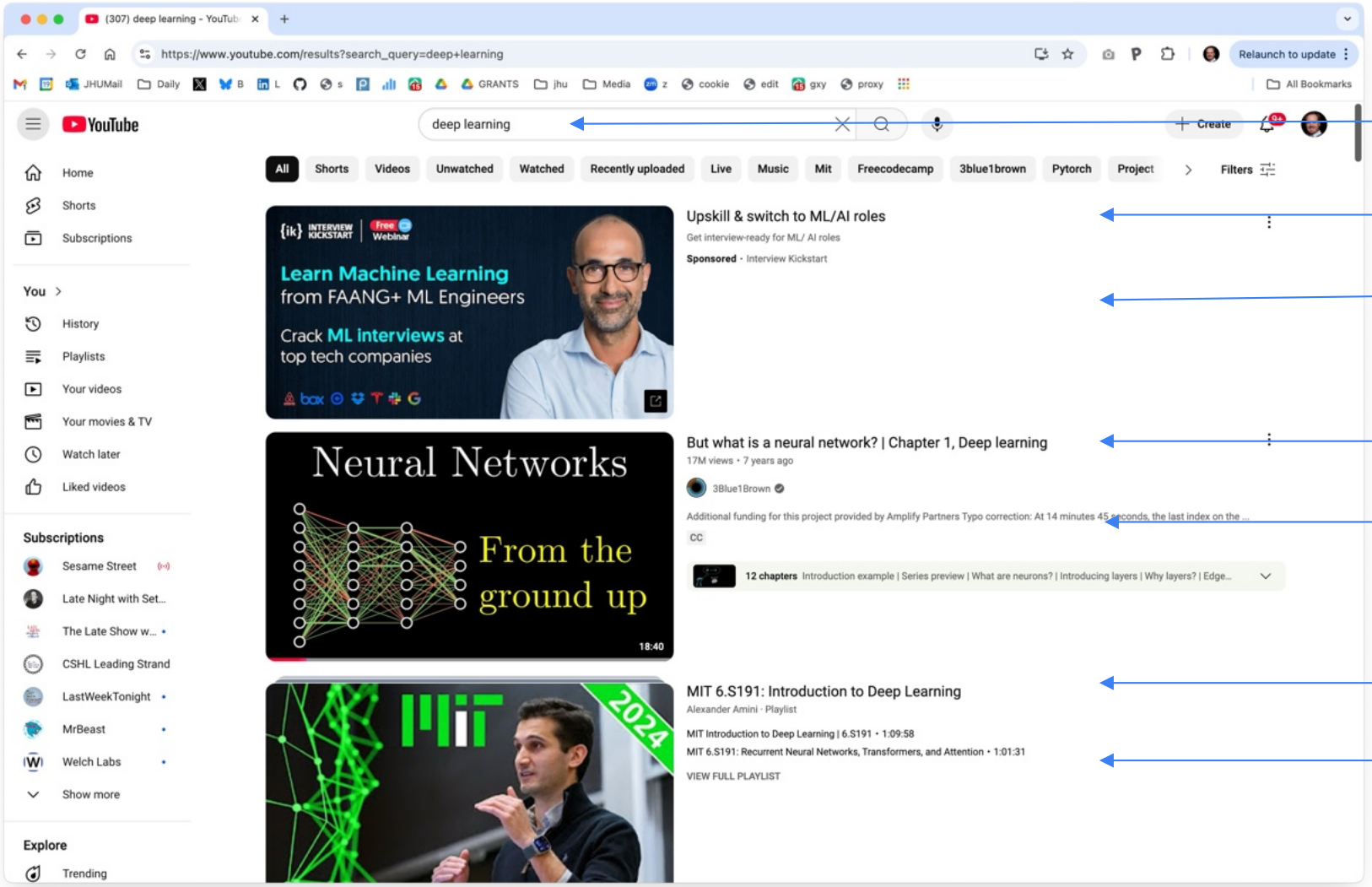
Output of attention head 1



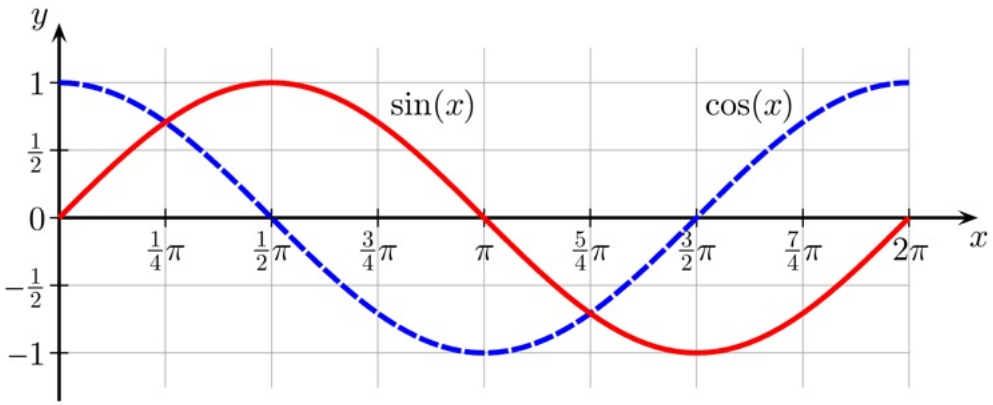
Output of attention head 2

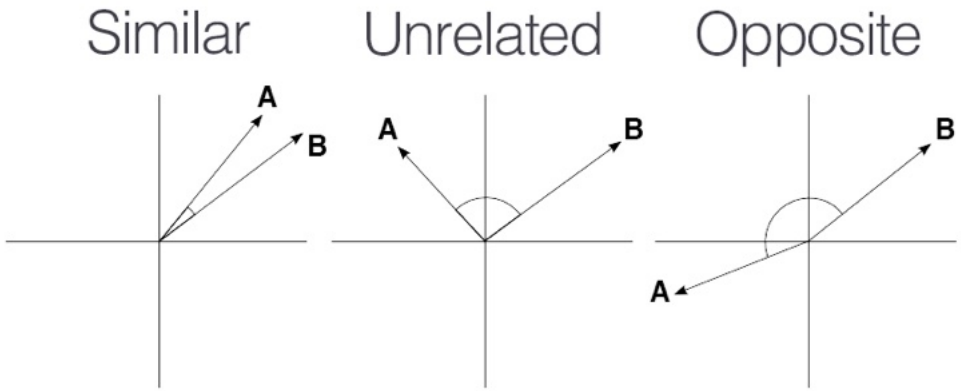


Output of attention head 3


(307) deep learning - YouTube
https://www.youtube.com/results?search_query=deep+learning
Relaunch to update
YouTube
deep learning
Create
Home
Shorts
Subscriptions
You
History
Playlists
Your videos
Your movies & TV
Watch later
Liked videos
Subscriptions
Sesame Street
Late Night with Set...
The Late Show w...
CSHL Leading Strand
LastWeekTonight
MrBeast
Welch Labs
Show more
Explore
Trending
All
Shorts
Videos
Unwatched
Watched
Recently uploaded
Live
Music
Mit
Freecodecamp
3blue1brown
Pytorch
Project
Filters
Learn Machine Learning
from FAANG+ ML Engineers
Crack ML interviews at
top tech companies
Upskill & switch to ML/AI roles
Get interview-ready for ML/ AI roles
Sponsored · Interview Kickstart
Neural Networks
From the
ground up
18:40
But what is a neural network? | Chapter 1, Deep learning
17M views · 7 years ago
3Blue1Brown
Additional funding for this project provided by Amplify Partners Typo correction: At 14 minutes 45 seconds, the last index on the ...
CC
12 chapters Introduction example | Series preview | What are neurons? | Introducing layers | Why layers? | Edge...
2024
MIT 6.S191: Introduction to Deep Learning
Alexander Amini · Playlist
MIT Introduction to Deep Learning | 6.S191 · 1:09:58
MIT 6.S191: Recurrent Neural Networks, Transformers, and Attention · 1:01:31
VIEW FULL PLAYLIST
Query
Key1
Value1
Key2
Value2
Key3
Value3

Cosine Similarity



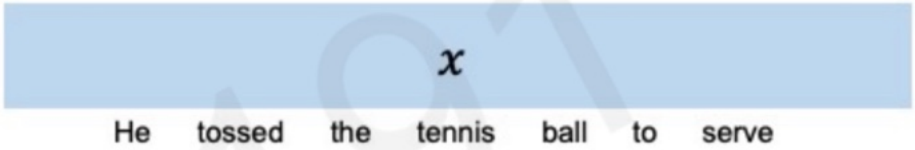


$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \cdot \sqrt{\sum_{i=1}^{n} B_i^2}},$$

where A_i and B_i are the ith components of vectors $\mathbf{A}$ and $\mathbf{B}$, respectively.

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.



1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract **features with high attention**

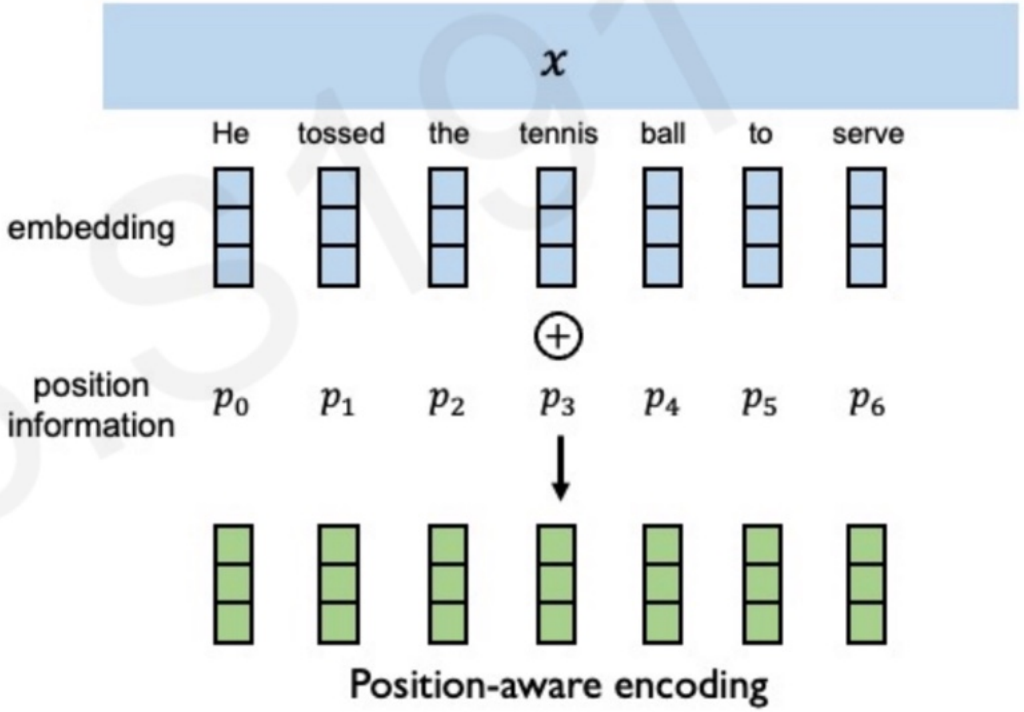
Data is fed in all at once! Need to encode position information to understand order.

MIT 6.S191

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract query, key, value for search
3. Compute attention weighting
4. Extract features with high attention



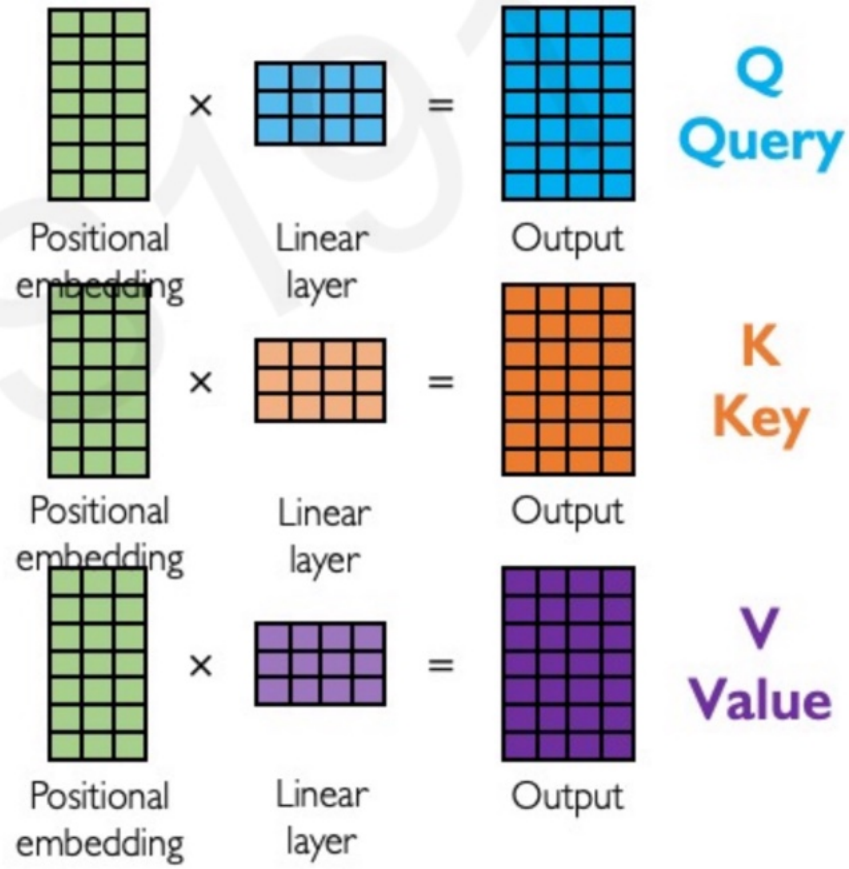
Data is fed in all at once! Need to encode position information to understand order.

MIT 6.S191

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute attention weighting
4. Extract features with high attention



MIT 6.S191

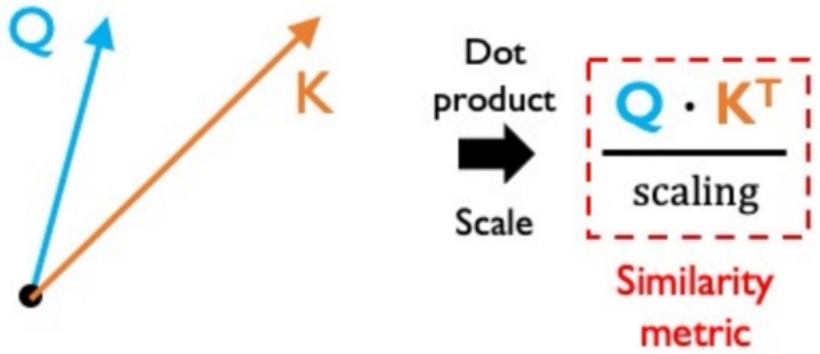
Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract features with high attention

Attention score: compute pairwise similarity between each query and key

How to compute similarity between two sets of features?



Also known as the "cosine similarity"

MIT 6.S191

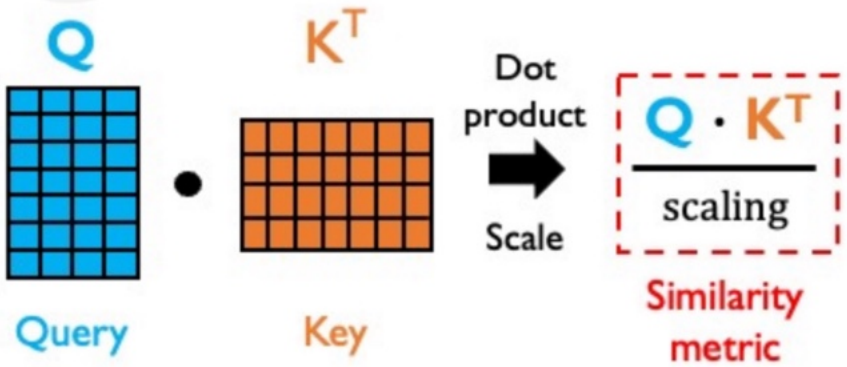
Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract features with high attention

Attention score: compute pairwise similarity between each query and key

How to compute similarity between two sets of features?



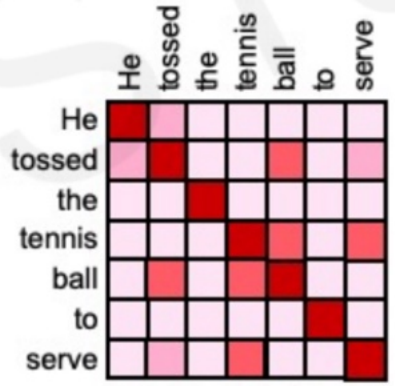
Also known as the "cosine similarity"

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract features with high attention

Attention weighting: where to attend to! How similar is the key to the query?



$$softmax\left(\frac{Q \cdot K^T}{scaling}\right)$$

Attention weighting

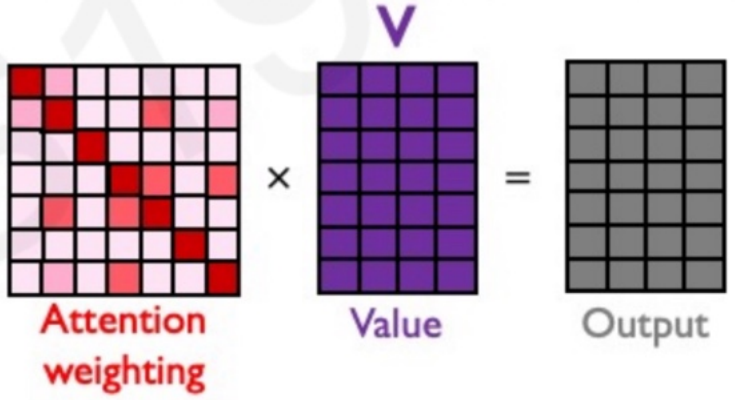
MIT 6.S191

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract **features with high attention**

Last step: self-attend to extract features



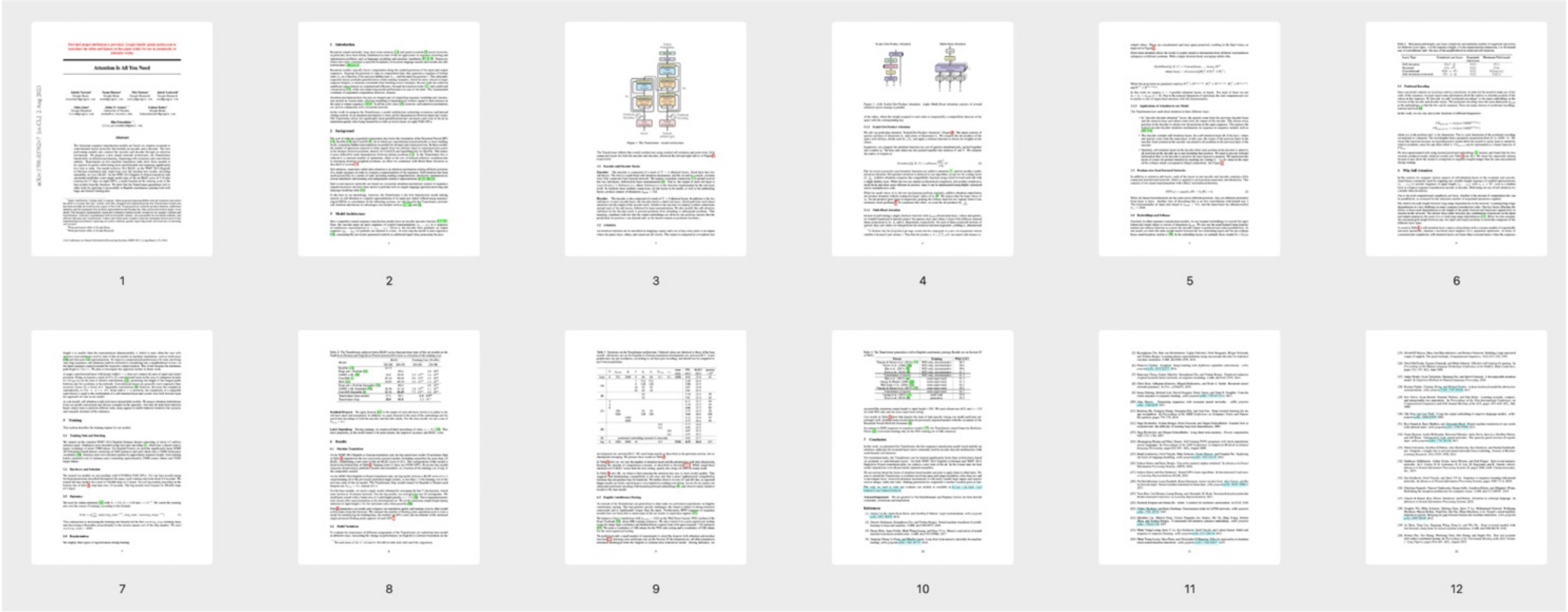
$$softmax\left(\frac{Q \cdot K^T}{scaling}\right) \cdot V = A(Q, K, V)$$

MIT 6.S191

Outline

1. Recap on ANNs and CNNs
2. The problem
3. Self-Attention
4. Transformers
5. Impact

The paper



Vaswani et al (2017) arXiv:1706.03762

Transformer: Multi-headed attention to encode and decode

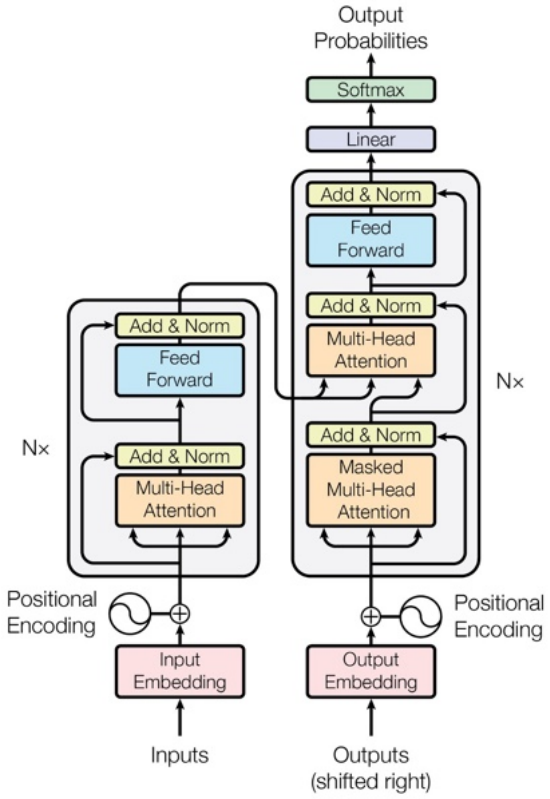


Figure 1: The Transformer - model architecture.

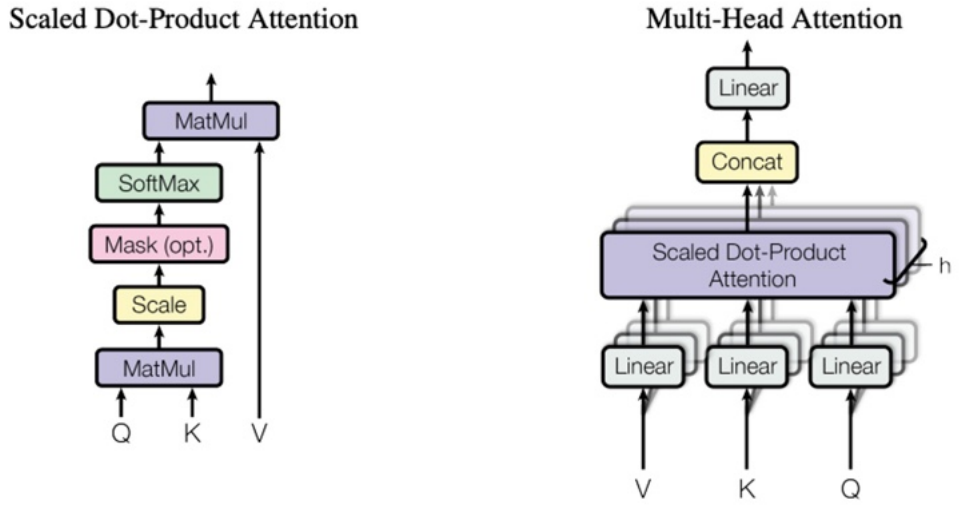


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

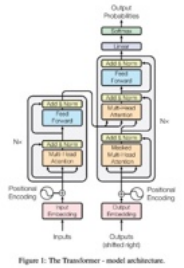
The application

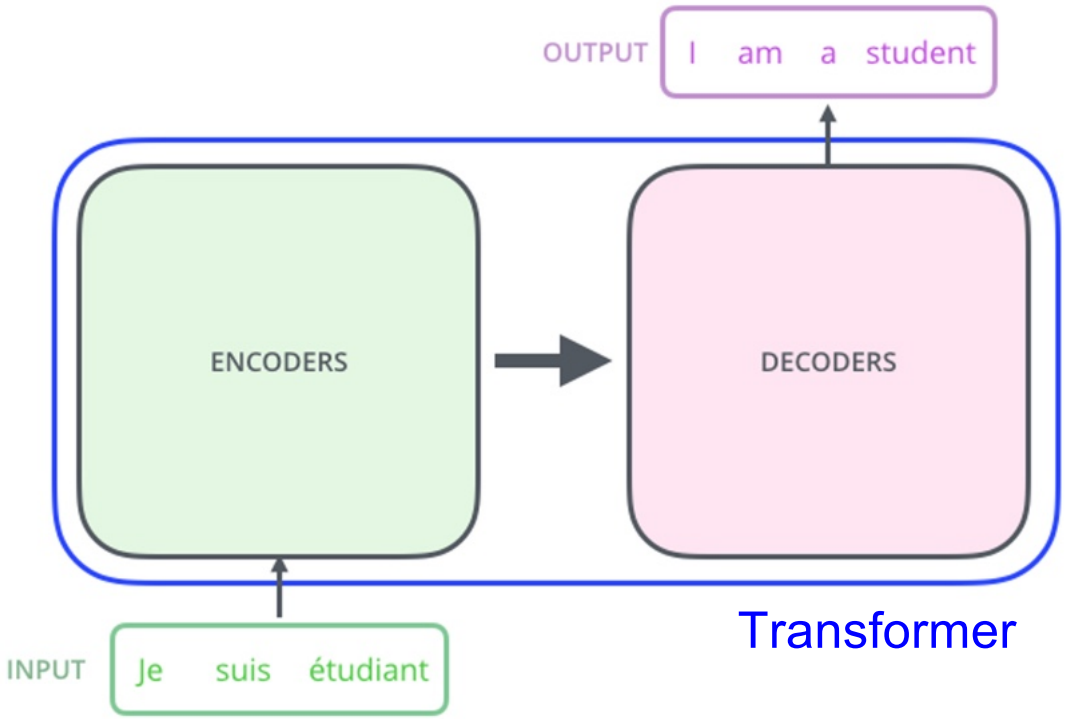


Trained with a huge collection of pairs of sentences in french & english
Measure accuracy using “BLEU” scores to gold standard (higher is better)
Use a “reasonable” amount of GPU time for training (<1 week on 8 GPUs)

https://jalammar.github.io/illustrated-transformer/

The architecture

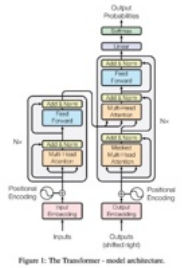


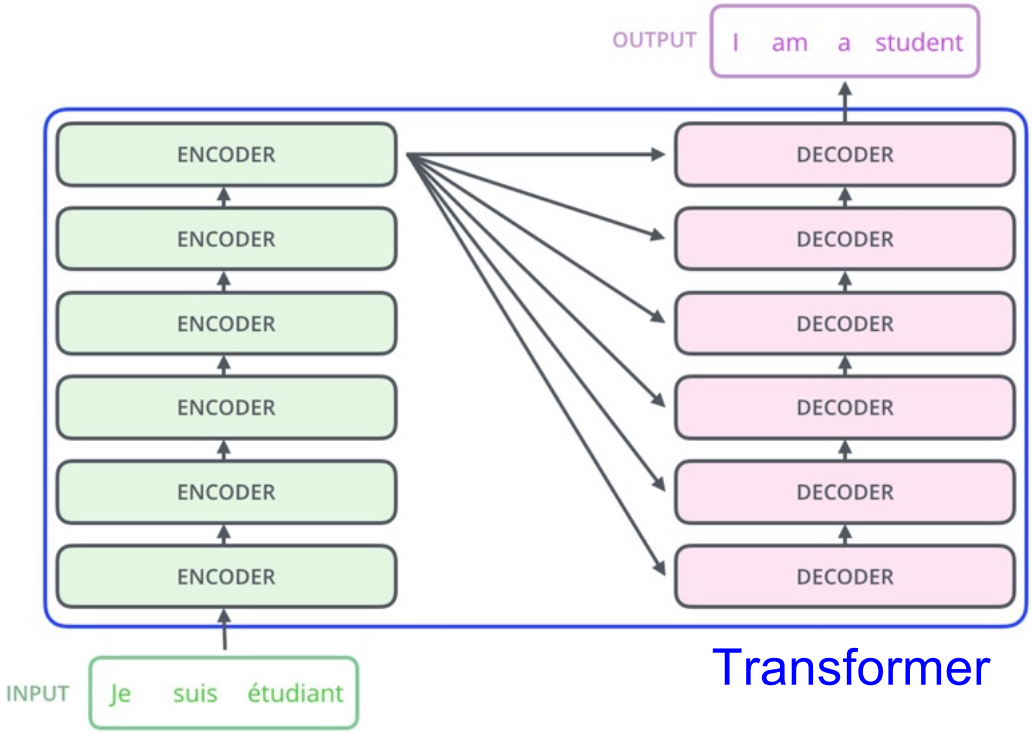


Transformer is composed of an input encoder followed by a output decoder

https://jalammar.github.io/illustrated-transformer/

The architecture

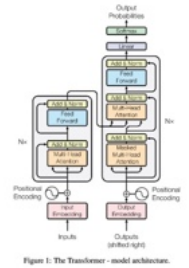


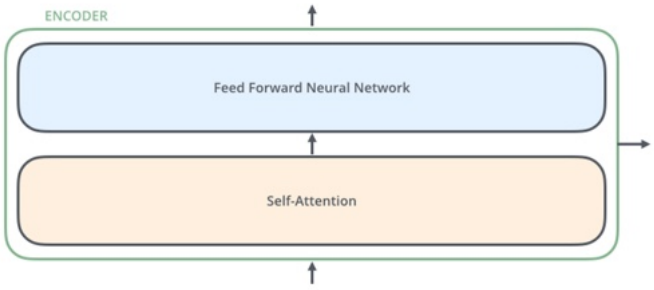


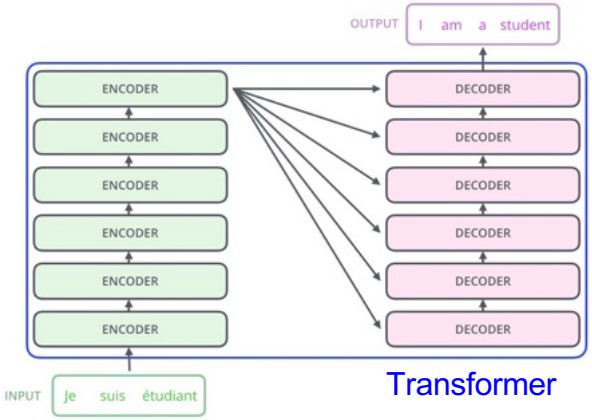
Encoder transforms sentences in language A into a highly abstract embedded space
Decoders progressively decodes highly abstract embedded space into language B

https://jalammar.github.io/illustrated-transformer/

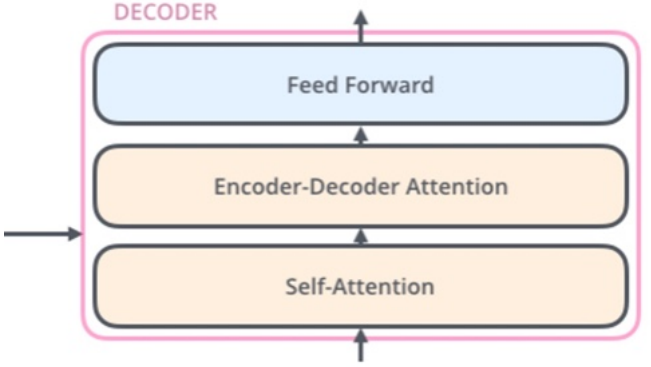
The architecture







Transformer

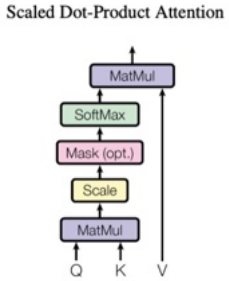


Encoders & Decoder layers use self-attention and simple feed forward networks

- Attention is all you need!

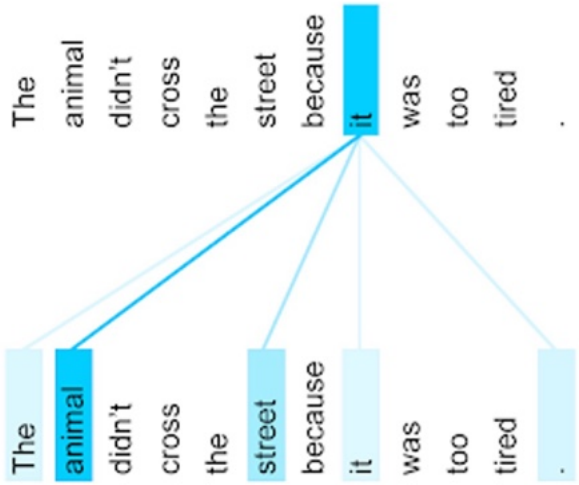
https://jalammar.github.io/illustrated-transformer/

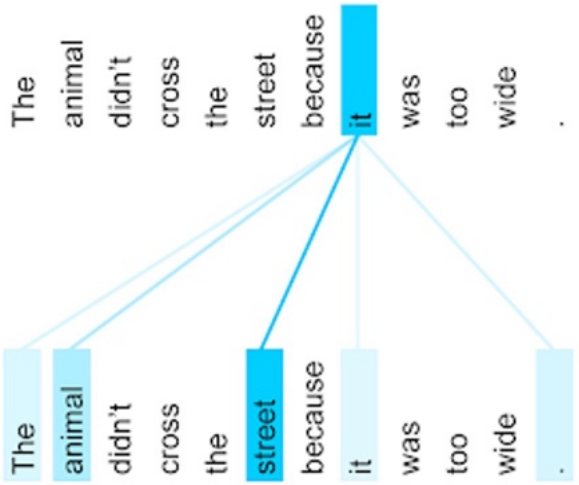
Self-attention



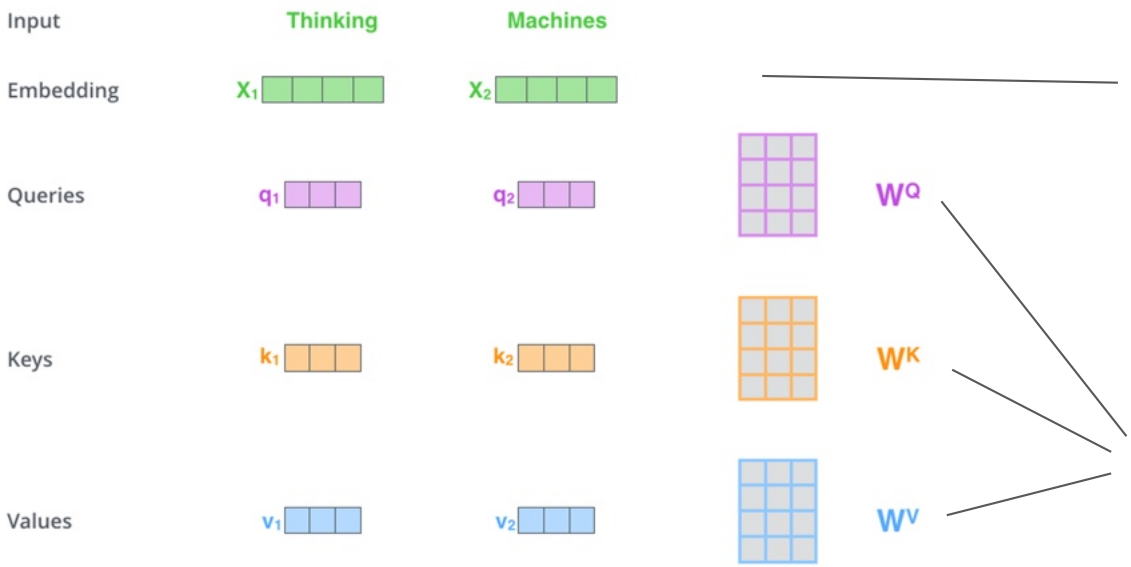
The animal didn't cross the street because it was too tired.
L'animal n'a pas traversé la rue parce qu'il était trop fatigué.

The animal didn't cross the street because it was too wide.
L'animal n'a pas traversé la rue parce qu'elle était trop large.





https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/



Embeddings are learned to project similar words into similar directions (cat & dog; king & queen)

- Similar to AlexNet!

Weight matrices (“masks”) learned during training

Multiply embedded input value times WQ, WK, and WV to compute q, k, v

Query (Q): Represents the current token's intention to "seek information" from other tokens in the sequence.

- It asks the question, "Which parts of the input sequence are relevant to me?"

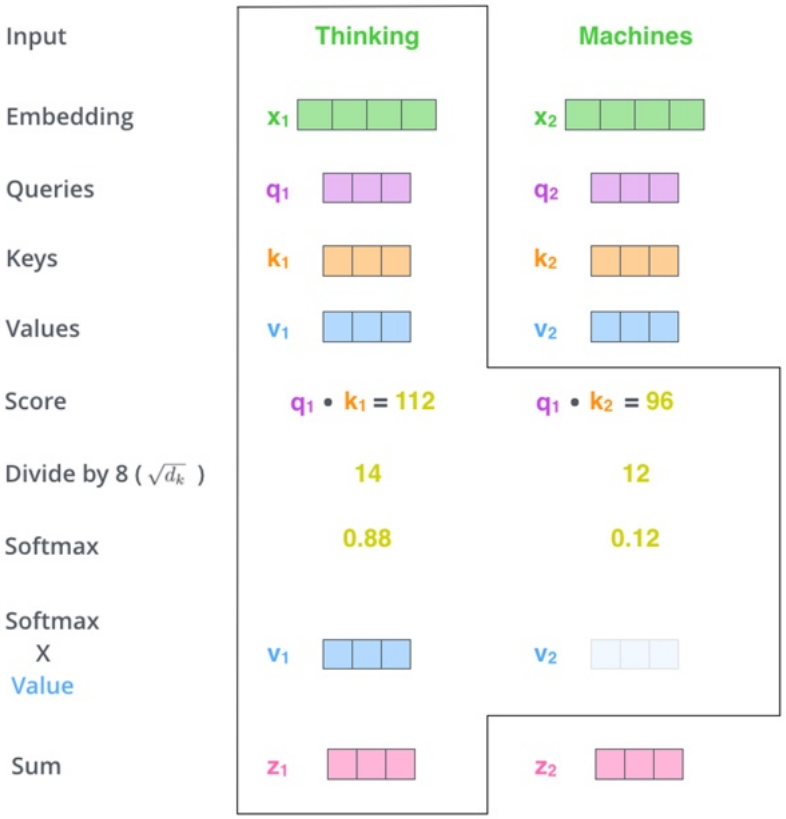
Key (K): Represents the content of the tokens against which the queries are compared.

- It acts like an "address" or "identifier" that the query uses to find relevant information.

Value (V): Contains the actual information that will be attended to. Once the attention mechanism determines which keys are most relevant to the query, the corresponding values are aggregated to form the final output.

https://jalammar.github.io/illustrated-transformer/

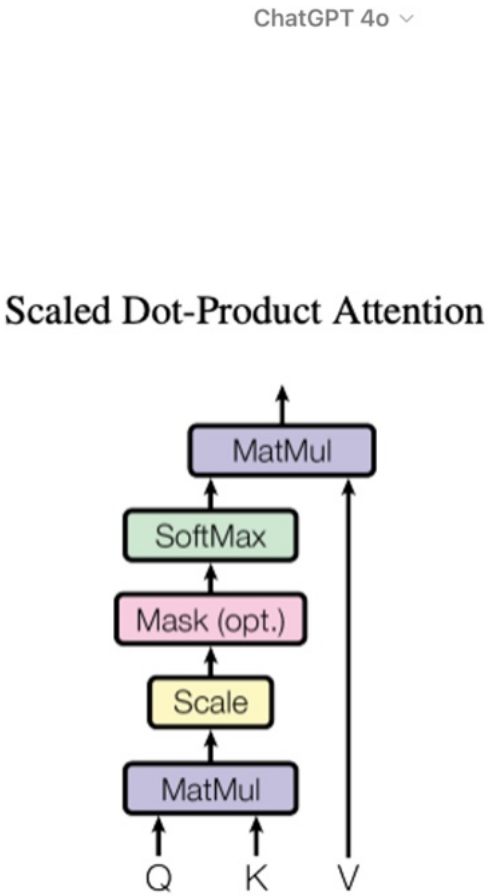
Attention computation



1. The score is calculated by taking the dot product (cosine similarity) of the **query vector** with the **key vector** of the respective word we're scoring.
2. The next steps are to divide the **scores** by 8 (the square root of the dimension of the key vectors used in the paper – 64). This leads to having more stable gradients.
3. Softmax normalizes the **scores** so they're all positive and add up to 1. This softmax **score** determines how much each word will be expressed at this position. ("Thinking" is mostly "Thinking" but a little "Machines")
4. Multiply each value vector by the softmax score to keep values of the word(s) we want to **focus** on, and drown-out irrelevant words
5. Sum up the weighted value vectors to produces the **output** of the self-attention layer at this position (for the first word). "Thinking" becomes "Thinking-machines"

https://jalammar.github.io/illustrated-transformer/

ChatGPT 4o



what is the softmax of the vector [14,12]

To compute the softmax of the vector $[14, 12]$, we can follow these steps:

1. **Compute the exponential** of each element in the vector.
2. **Sum** all the exponentials computed in the first step.
3. **Divide** each exponential by the sum from the second step to get the softmax values.

Let's calculate this:

1. Exponentials:

$$e^{14} \approx 1202604.28$$

$$e^{12} \approx 162754.79$$

2. Sum of exponentials:

$$1202604.28 + 162754.79 = 1365359.07$$

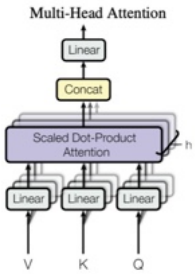
3. Softmax values:

$$\text{Softmax of } 14 = \frac{1202604.28}{1365359.07} \approx 0.881$$

$$\text{Softmax of } 12 = \frac{162754.79}{1365359.07} \approx 0.119$$

So, the softmax of the vector $[14, 12]$ is approximately $[0.881, 0.119]$.

Multi-headed Attention



1) This is our input sentence*

2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using the resulting Q/K/V matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

Thinking Machines

X

W_0^Q W_0^K W_0^V

Q_0 K_0 V_0

Z_0

W^O

Z

W_1^Q W_1^K W_1^V

Q_1 K_1 V_1

Z_1

* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

R

...

W_7^Q W_7^K W_7^V

Q_7 K_7 V_7

Z_7

Similar to having multiple convolutional filters

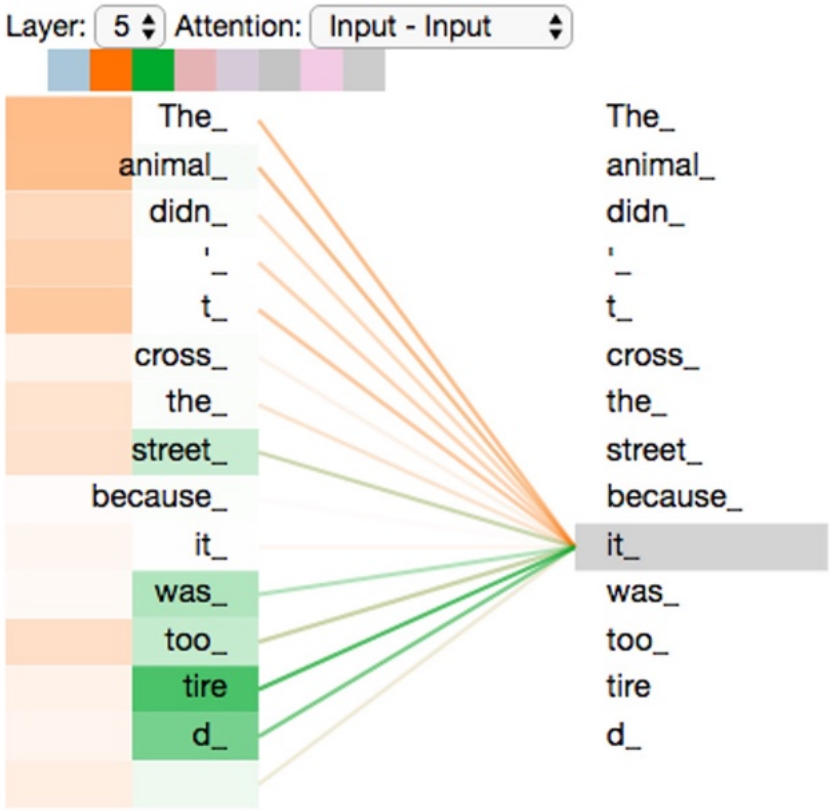
https://jalammar.github.io/illustrated-transformer/

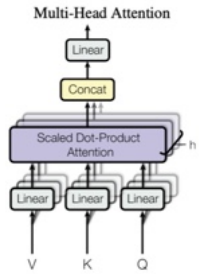
Multi-headed Attention

The orange attention head “knows” “it” refers to animal

The green attention head “knows” “it” refers to tired

Added together we form a new vector representing the concept of “tired animal”





https://jalammar.github.io/illustrated-transformer/

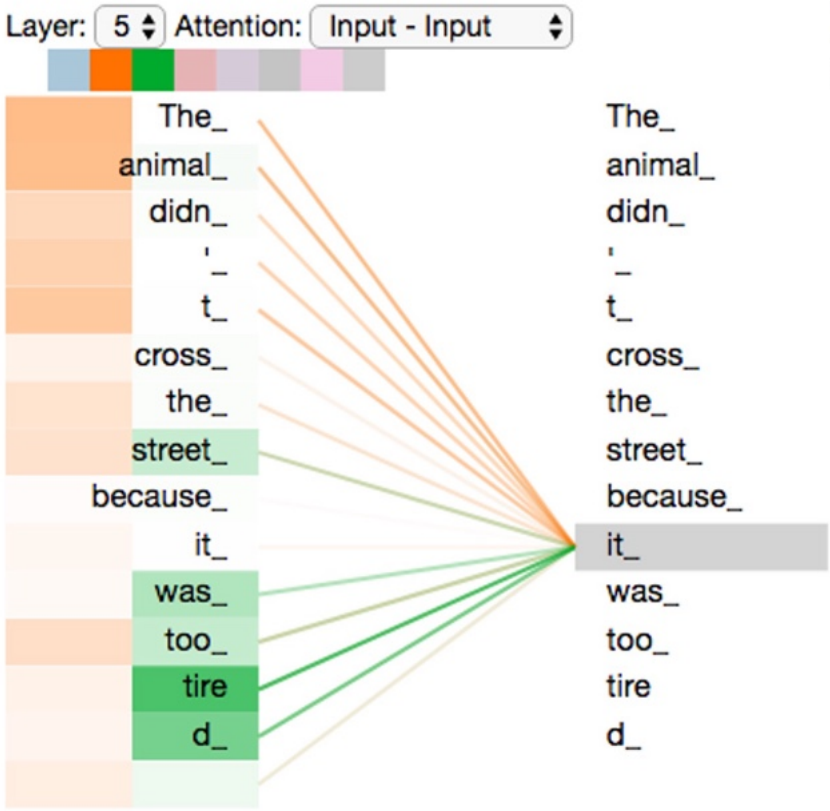
Multi-headed Attention

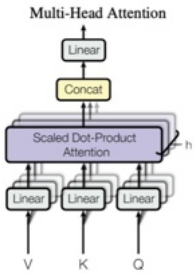
The orange attention head “knows” “it” refers to animal

The green attention head “knows” “it” refers to tired

Added together we form a new vector representing the concept of “tired animal”

* One head will be trained to look at previous word





https://jalammar.github.io/illustrated-transformer/

Decoding

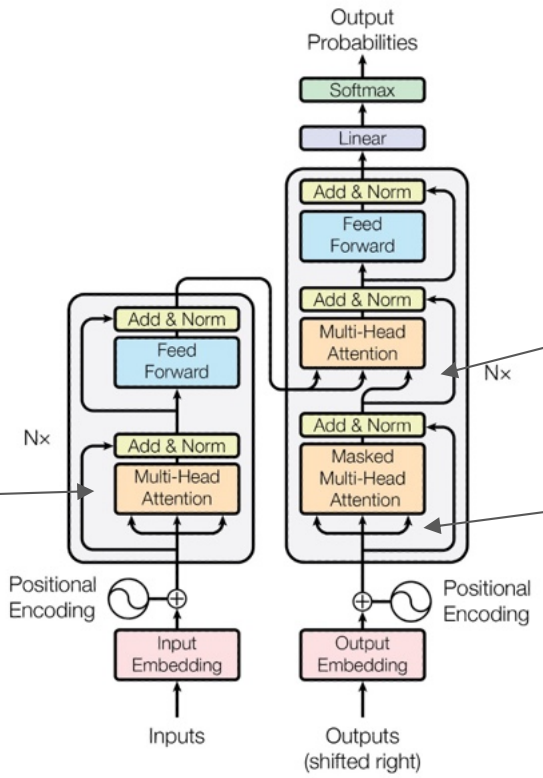


Figure 1: The Transformer - model architecture.

Magic! Joint attention between abstracted input language to abstracted output language

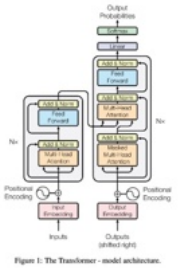
Self-attention of input stream

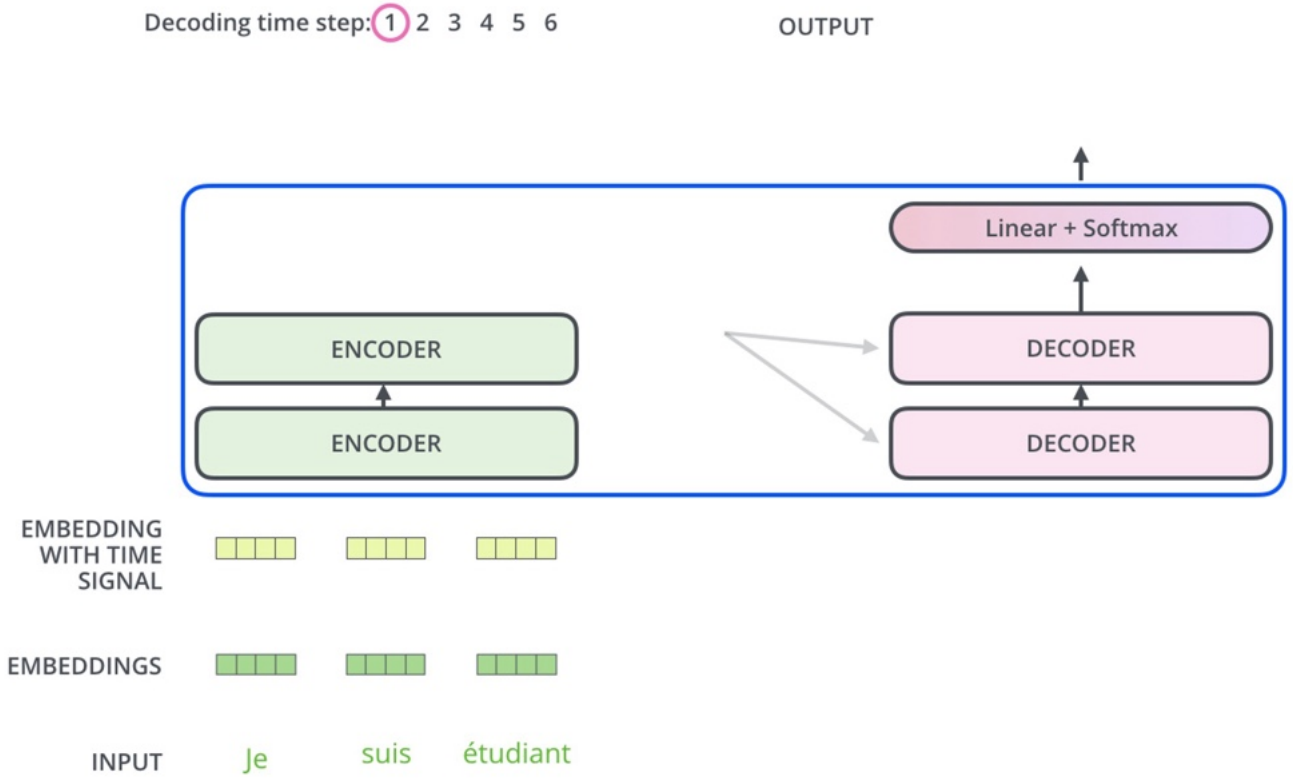
Self-attention of output stream

Encoder transforms sentences in language A into a highly abstract embedded space
Decoders progressively decodes highly abstract embedded space into language B

https://jalammar.github.io/illustrated-transformer/

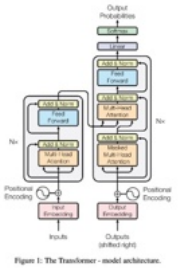
Decoding

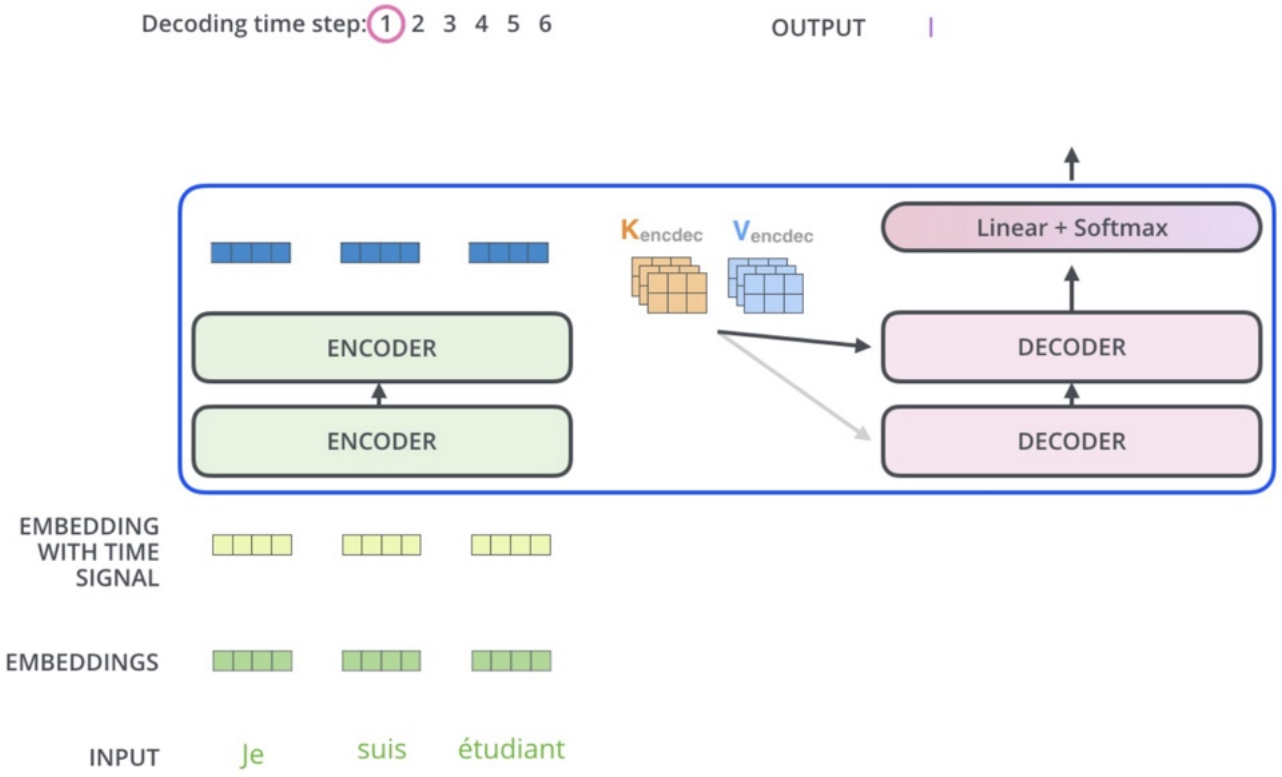




https://jalammar.github.io/illustrated-transformer/

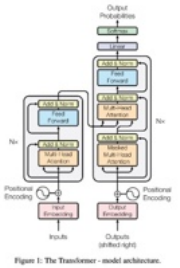
Decoding

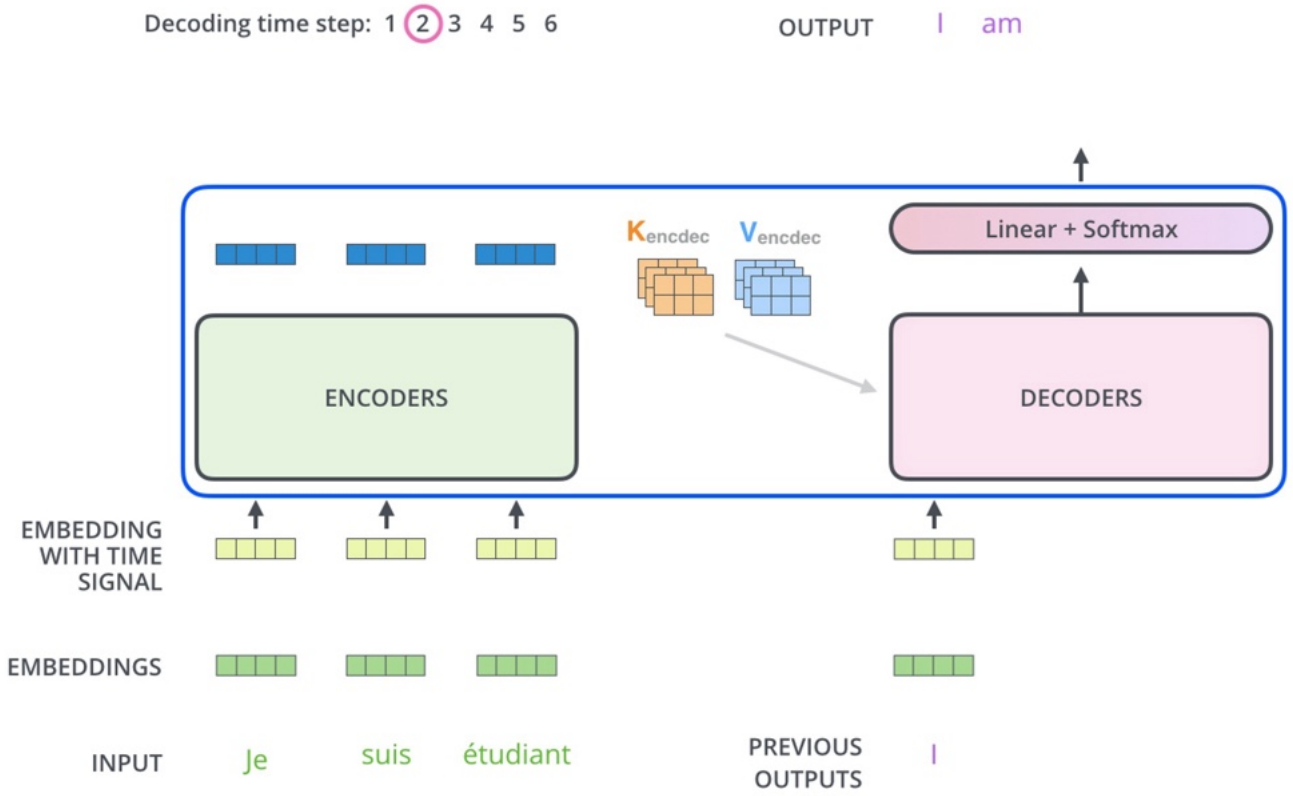




https://jalammar.github.io/illustrated-transformer/

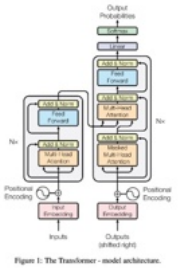
Decoding

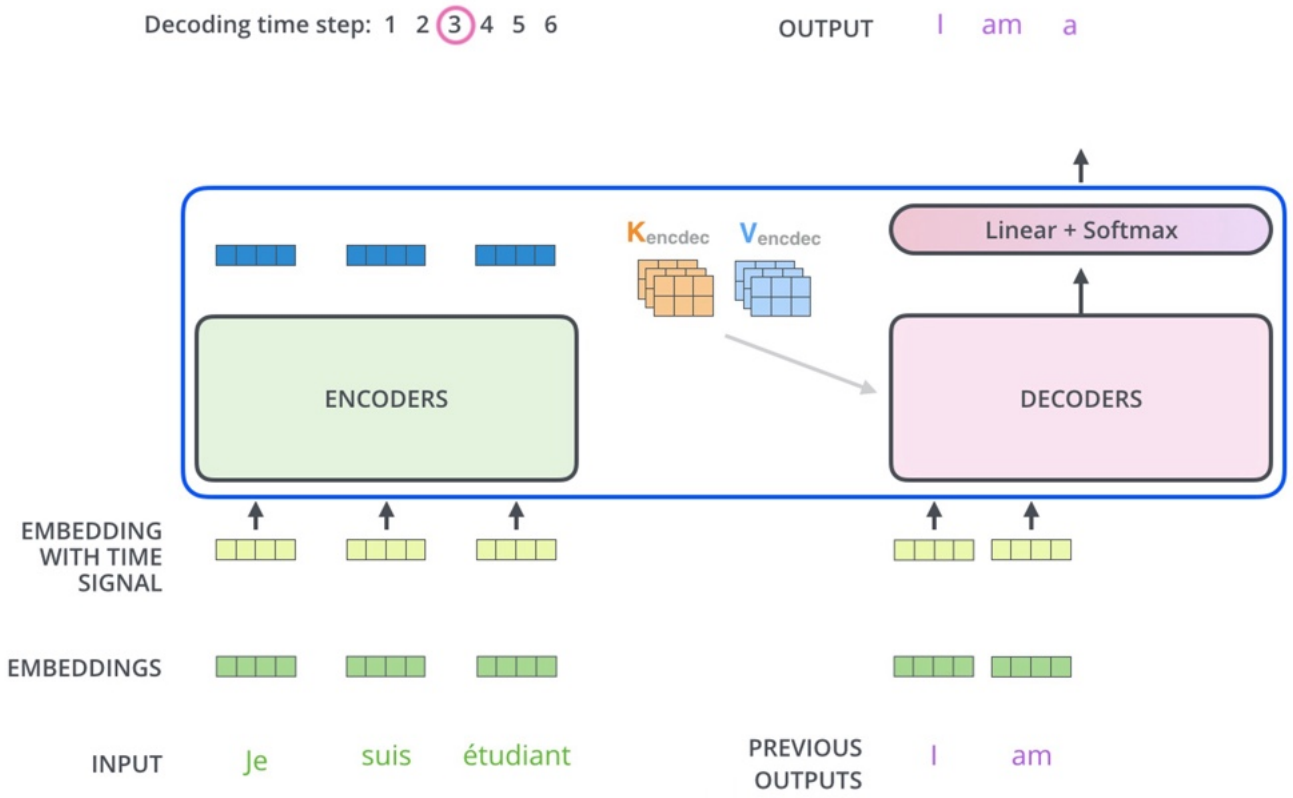




https://jalammar.github.io/illustrated-transformer/

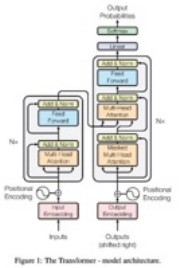
Decoding

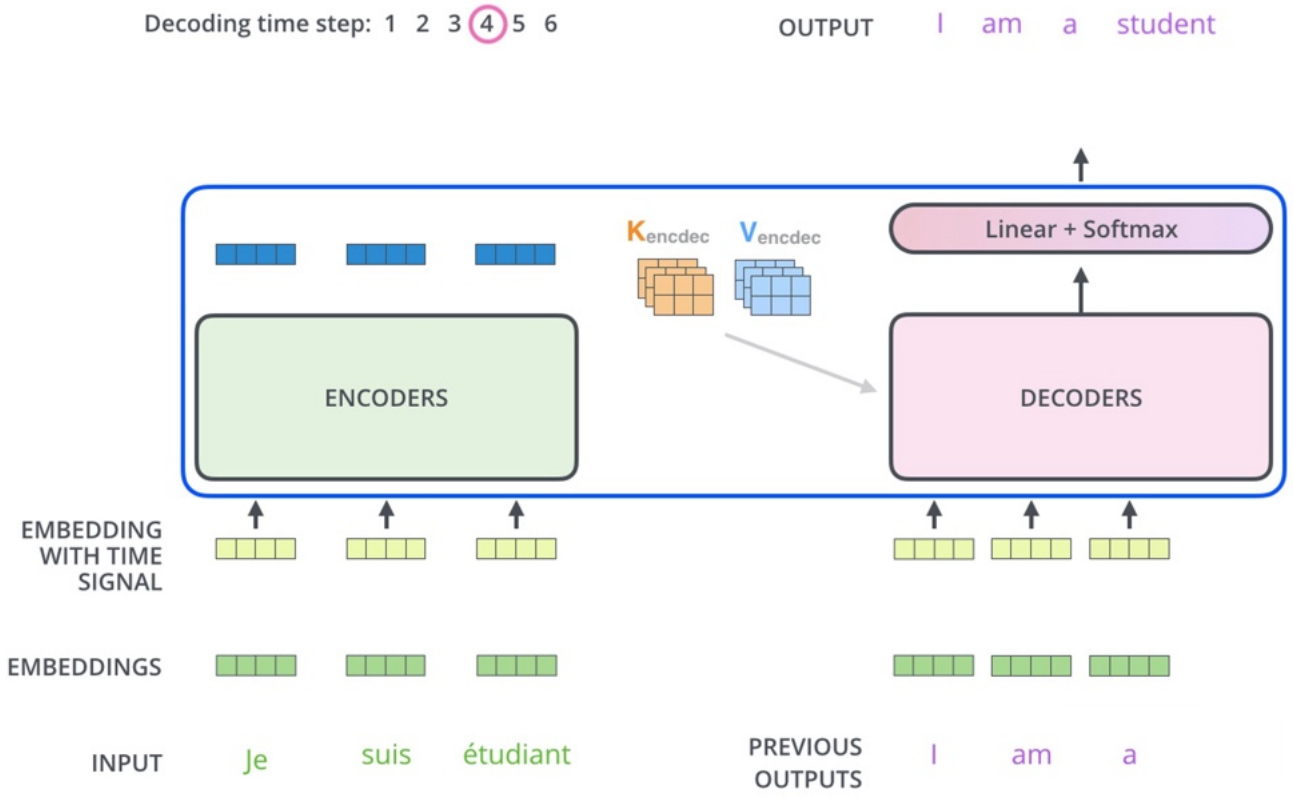




https://jalammar.github.io/illustrated-transformer/

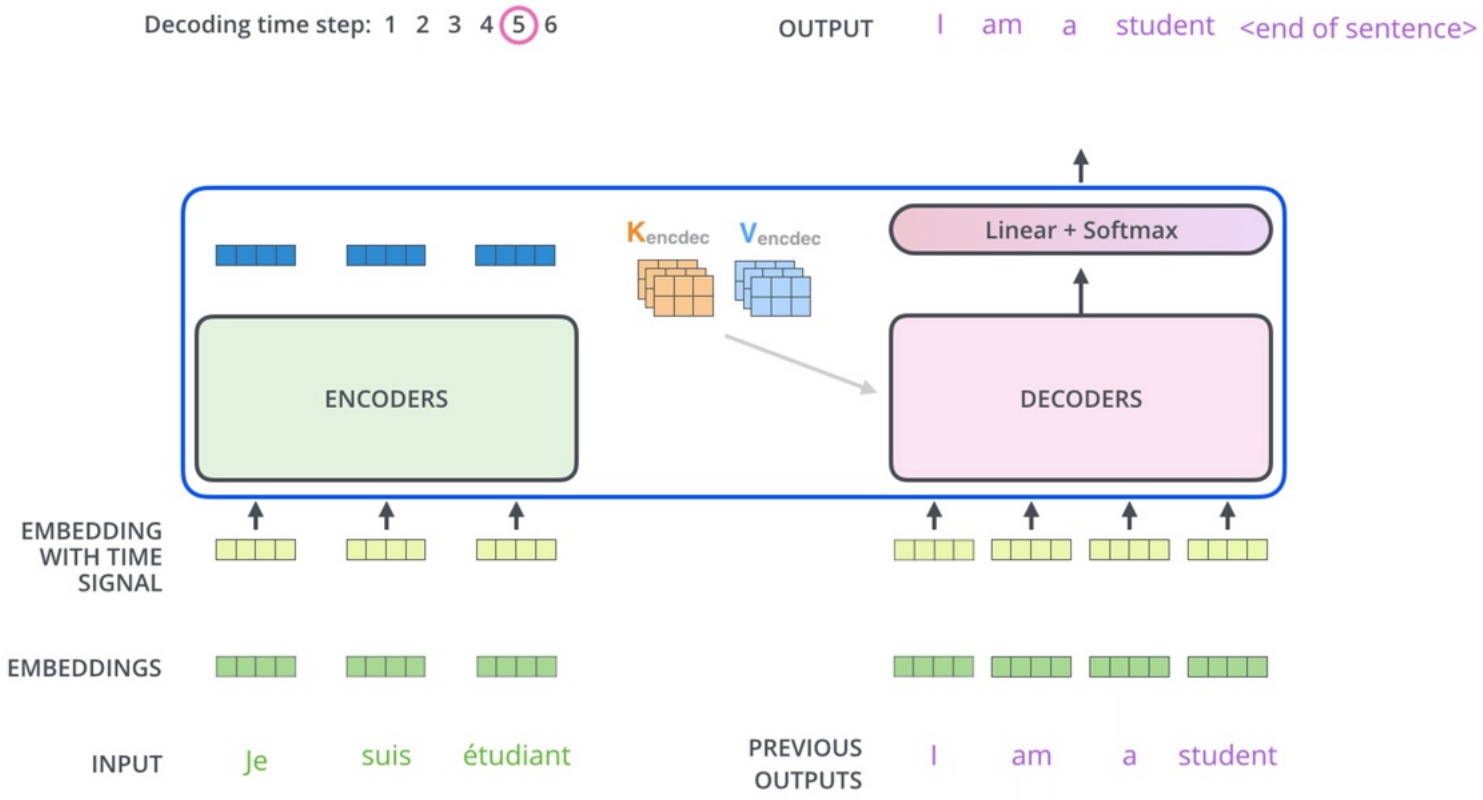
Decoding





https://jalammar.github.io/illustrated-transformer/

Decoding



https://jalammar.github.io/illustrated-transformer/

Decoding back into words

Very similar to last few layers of AlexNet!

Which word in our vocabulary is associated with this index?

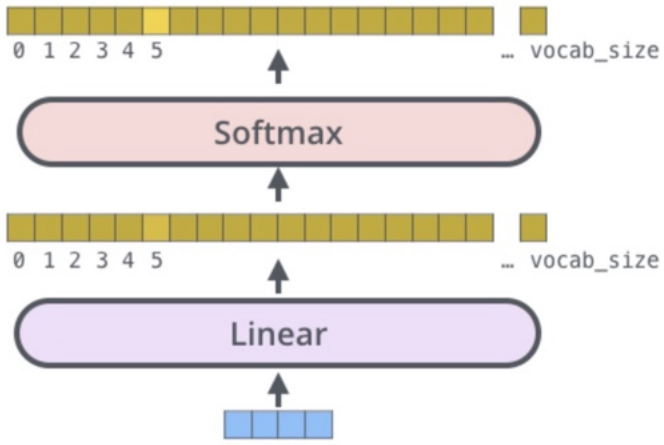
am

Which word is this?

Get the index of the cell with the highest value (argmax)

5

Pick the most probable



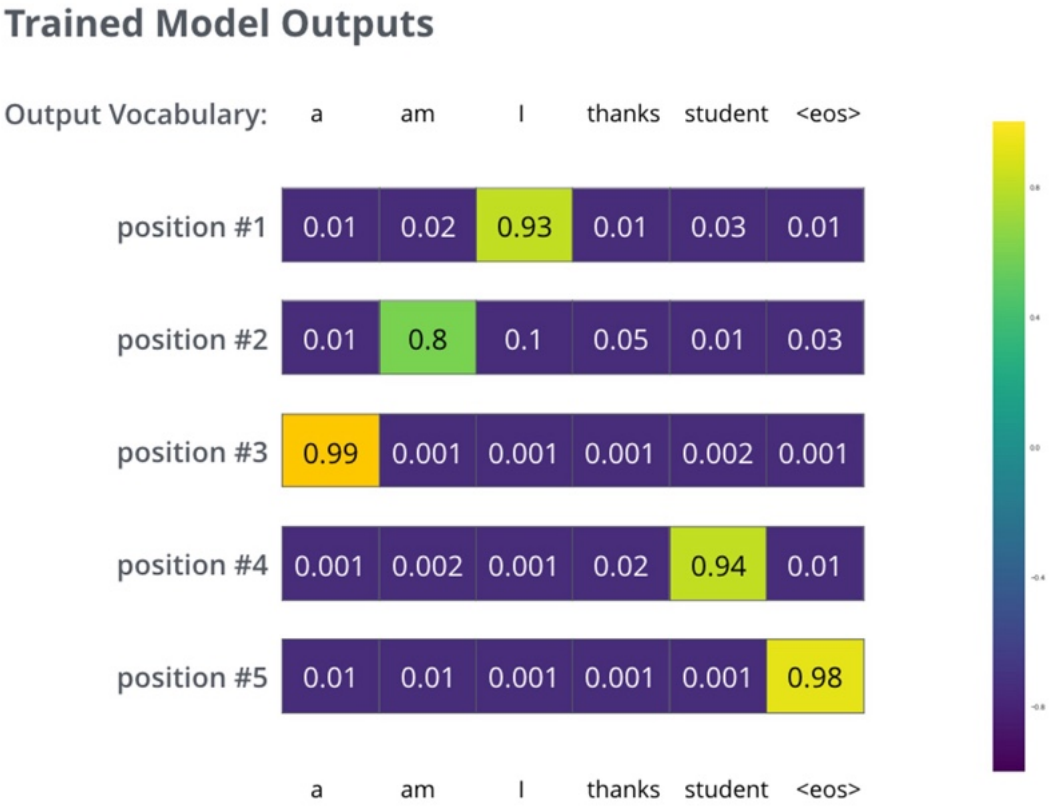
Prob. of being each term

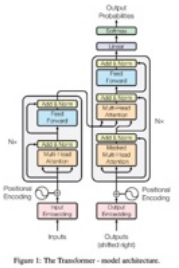
Score for being a term

Abstract embedding

https://jalammar.github.io/illustrated-transformer/

Complete output





Greedy decoding:

- Always pick the word with the highest probability

Beam search:

- run the model twice: once assuming the first output position was ‘I’, and another as ‘a’, maintain both decodings

Perplexity:

- Introduce randomness into the output

https://jalammar.github.io/illustrated-transformer/

Transformer: Multi-headed attention to encode and decode

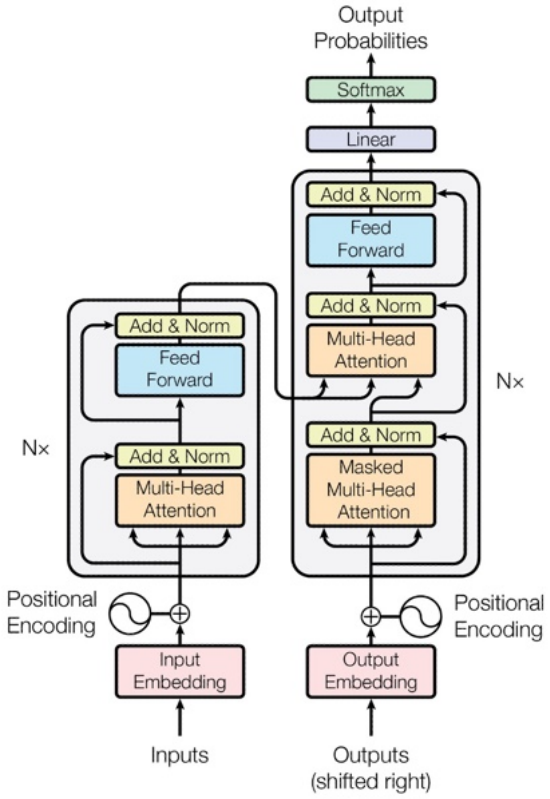


Figure 1: The Transformer - model architecture.

The Transformer uses multi-head attention in three different ways:

"Encoder self-attention": In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.

"Encoder-decoder attention": The queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence.

"Self-attention in the decoder": Allows each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections.

Performance

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

More accurate and 100 to 1000 times faster!

Outline

1. Recap on ANNs and CNNs
2. The problem
3. Self-Attention and Transformers
4. Impact

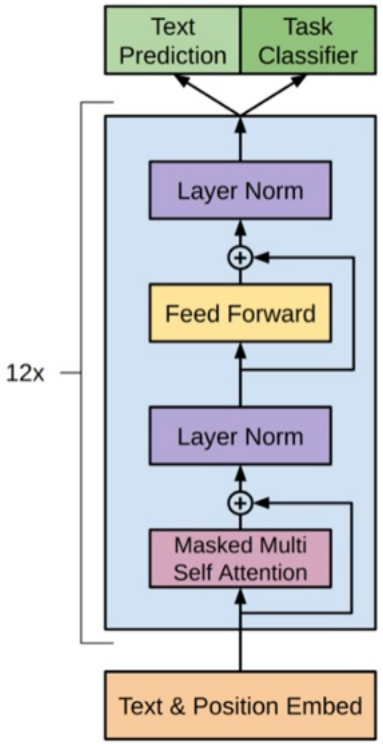
GPT-1

Improving Language Understanding by Generative Pre-Training

Alec Radford OpenAI alec@openai.com
Karthik Narasimhan OpenAI karthikn@openai.com
Tim Salimans OpenAI tim@openai.com
Ilya Sutskever OpenAI ilyasu@openai.com

Abstract

Natural language understanding comprises a wide range of diverse tasks such as textual entailment, question answering, semantic similarity assessment, and document classification. Although large unlabeled text corpora are abundant, labeled data for learning these specific tasks is scarce, making it challenging for discriminatively trained models to perform adequately. We demonstrate that large gains on these tasks can be realized by *generative pre-training* of a language model on a diverse corpus of unlabeled text, followed by *discriminative fine-tuning* on each specific task. In contrast to previous approaches, we make use of task-aware input transformations during fine-tuning to achieve effective transfer while requiring minimal changes to the model architecture. We demonstrate the effectiveness of our approach on a wide range of benchmarks for natural language understanding. Our general task-agnostic model outperforms discriminatively trained models that use architectures specifically crafted for each task, significantly improving upon the state of the art in 9 out of the 12 tasks studied. For instance, we achieve absolute improvements of 8.9% on commonsense reasoning (Stories Cloze Test), 5.7% on question answering (RACE), and 1.5% on textual entailment (MultiNLI).



GPT-2

Language Models are Unsupervised Multitask Learners

Alec Radford [*1] Jeffrey Wu [*1] Rewon Child [1] David Luan [1] Dario Amodei [**1] Ilya Sutskever [**1]

Abstract

Natural language processing tasks, such as question answering, machine translation, reading comprehension, and summarization, are typically approached with supervised learning on task-specific datasets. We demonstrate that language models begin to learn these tasks without any explicit supervision when trained on a new dataset of millions of webpages called WebText. When conditioned on a document plus questions, the answers generated by the language model reach 55 F1 on the CoQA dataset - matching or exceeding the performance of 3 out of 4 baseline systems without using the 127,000+ training examples. The capacity of the language model is essential to the success of zero-shot task transfer and increasing it improves performance in a log-linear fashion across tasks. Our largest model, GPT-2, is a 1.5B parameter Transformer that achieves state of the art results on 7 out of 8 tested language modeling datasets in a zero-shot setting but still underfits WebText. Samples from the model reflect these improvements and contain coherent paragraphs of text. These findings suggest a promising path towards building language processing systems which learn to perform tasks from their naturally occurring demonstrations.

1. Introduction

Machine learning systems now excel (in expectation) at tasks they are trained for by using a combination of large datasets, high-capacity models, and supervised learning (Krizhevsky et al., 2012) (Sutskever et al., 2014) (Amodei et al., 2016). Yet these systems are brittle and sensitive to slight changes in the data distribution (Recht et al., 2018) and task specification (Kirkpatrick et al., 2017). Current systems are better characterized as narrow experts rather than competent generalists. We would like to move towards more general systems which can perform many tasks – eventually without the need to manually create and label a training dataset for each one.

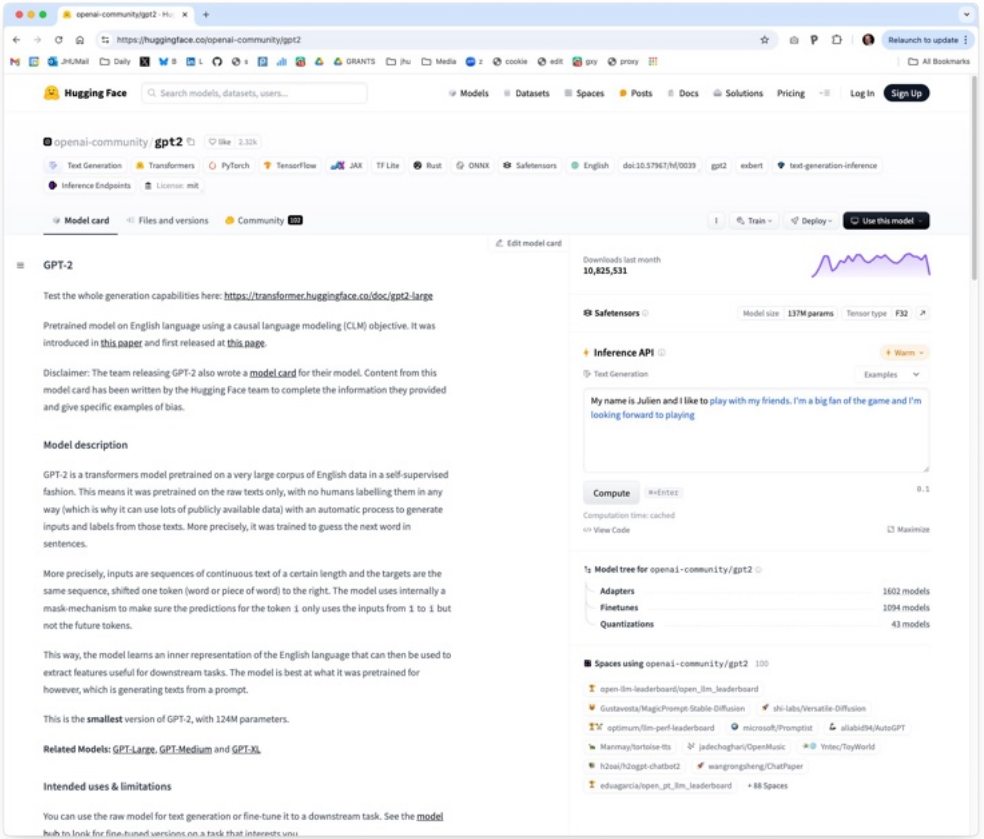
The dominant approach to creating ML systems is to collect a dataset of training examples demonstrating correct behavior for a desired task, train a system to imitate these behaviors, and then test its performance on independent and identically distributed (IID) held-out examples. This has served well to make progress on narrow experts. But the often erratic behavior of captioning models (Lake et al., 2017), reading comprehension systems (Jia & Liang, 2017), and image classifiers (Alcorn et al., 2018) on the diversity and variety of possible inputs highlights some of the shortcomings of this approach.

Our suspicion is that the prevalence of single task training on single domain datasets is a major contributor to the lack of generalization observed in current systems. Progress towards robust systems with current architectures is likely to require training and measuring performance on a wide range of domains and tasks. Recently, several benchmarks have been proposed such as GLUE (Wang et al., 2018) and decaNLP (McCann et al., 2018) to begin studying this.

Multitask learning (Caruana, 1997) is a promising framework for improving general performance. However, multitask training in NLP is still nascent. Recent work reports modest performance improvements (Yogatama et al., 2019) and the two most ambitious efforts to date have trained on a total of 10 and 17 `(dataset, objective)` pairs respectively (McCann et al., 2018) (Bowman et al., 2018). From a meta-learning perspective, each `(dataset, objective)` pair is a single training example sampled from the distribution of datasets and objectives. Current ML systems need hundreds to thousands of examples to induce functions which generalize well. This suggests that multitask training many need just as many effective training pairs to realize its promise with current approaches. It will be very difficult to continue to scale the creation of datasets and the design of objectives to the degree that may be required to brute force our way there with current techniques. This motivates exploring additional setups for performing multitask learning.

The current best performing systems on language tasks

[*,**]Equal contribution [1]OpenAI, San Francisco, California, United States. Correspondence to: Alec Radford <alec@openai.com>.





https://www.youtube.com/watch?v=UZDiGooFs54

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin **Ming-Wei Chang** **Kenton Lee** **Kristina Toutanova**
Google AI Language
{jacobdevlin,mingweichang,kentonl,kristout}@google.com

Abstract

We introduce a new language representation model called **BERT**, which stands for **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications.

BERT is conceptually simple and empirically powerful. It obtains new state-of-the-art results on eleven natural language processing tasks, including pushing the GLUE score to 80.5% (7.7% point absolute improvement), MultiNLI accuracy to 86.7% (4.6% absolute improvement), SQuAD v1.1 question answering Test F1 to 93.2 (1.5 point absolute improvement) and SQuAD v2.0 Test F1 to 83.1 (5.1 point absolute improvement).

1 Introduction

Language model pre-training has been shown to be effective for improving many natural language processing tasks (Dai and Le, 2015; Peters et al., 2018a; Radford et al., 2018; Howard and Ruder, 2018). These include sentence-level tasks such as natural language inference (Bowman et al., 2015; Williams et al., 2018) and paraphrasing (Dolan and Brockett, 2005), which aim to predict the relationships between sentences by analyzing them holistically, as well as token-level tasks such as

There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal task-specific parameters, and is trained on the downstream tasks by simply fine-tuning *all* pre-trained parameters. The two approaches share the same objective function during pre-training, where they use unidirectional language models to learn general language representations.

We argue that current techniques restrict the power of the pre-trained representations, especially for the fine-tuning approaches. The major limitation is that standard language models are unidirectional, and this limits the choice of architectures that can be used during pre-training. For example, in OpenAI GPT, the authors use a left-to-right architecture, where every token can only attend to previous tokens in the self-attention layers of the Transformer (Vaswani et al., 2017). Such restrictions are sub-optimal for sentence-level tasks, and could be very harmful when applying fine-tuning based approaches to token-level tasks such as question answering, where it is crucial to incorporate context from both directions.

In this paper, we improve the fine-tuning based approaches by proposing BERT: **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. BERT alleviates the previously mentioned unidirectionality constraint by using a "masked language model" (MLM) pre-training objective, in-

BERT: Pre-training of Deep Bidirectional Transformers for ...

by J Devlin · 2018 · Cited by 108443 — We introduce a new **language** representation model

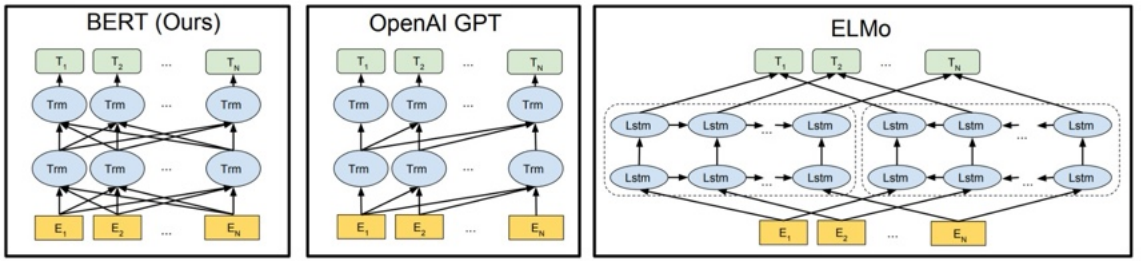
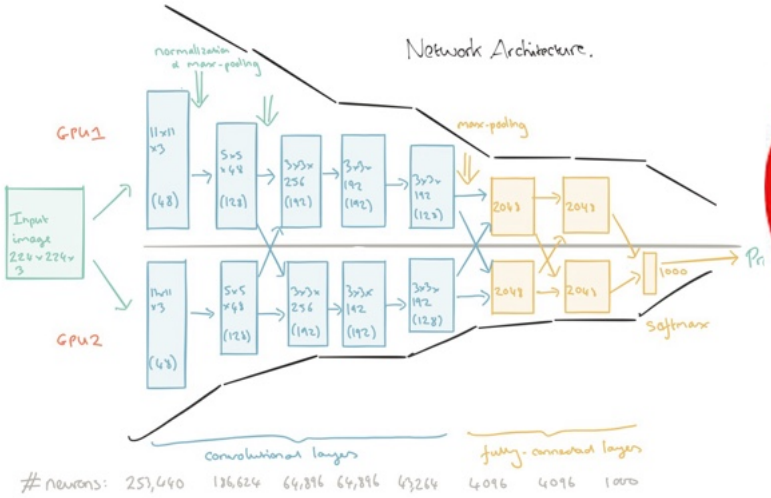


Figure 3: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks. Among the three, only BERT representations are jointly conditioned on both left and right context in all layers. In addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

3. Using BERT for Different NLP Tasks

- **Text Classification**: BERT can be used for tasks like sentiment analysis, spam detection, or topic classification. After fine-tuning, the `[CLS]` token's output (the special token at the start of the sequence) is passed through a classifier to predict the label of the text.
- **Named Entity Recognition (NER)**: BERT can be fine-tuned to recognize and classify named entities (e.g., names, organizations, dates) in text. Each token's output from BERT is passed through a classification layer to predict the entity label.
- **Question Answering**: For tasks like SQuAD (Stanford Question Answering Dataset), BERT can be used to find the answer to a question within a given passage. The model is fine-tuned to predict the start and end positions of the answer span in the passage.
- **Sentence Pair Classification**: Tasks like entailment (e.g., recognizing textual entailment) or semantic textual similarity can be handled by BERT. Two sentences are input, and BERT is fine-tuned to classify their relationship (e.g., entailment, contradiction, neutral).
- **Text Generation**: While BERT itself is not primarily a text generation model (since it's not designed to generate text sequentially), it can be adapted for certain text completion tasks or used in conjunction with other models for generation.





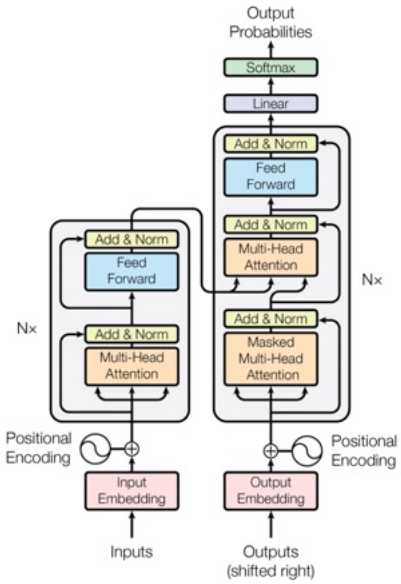
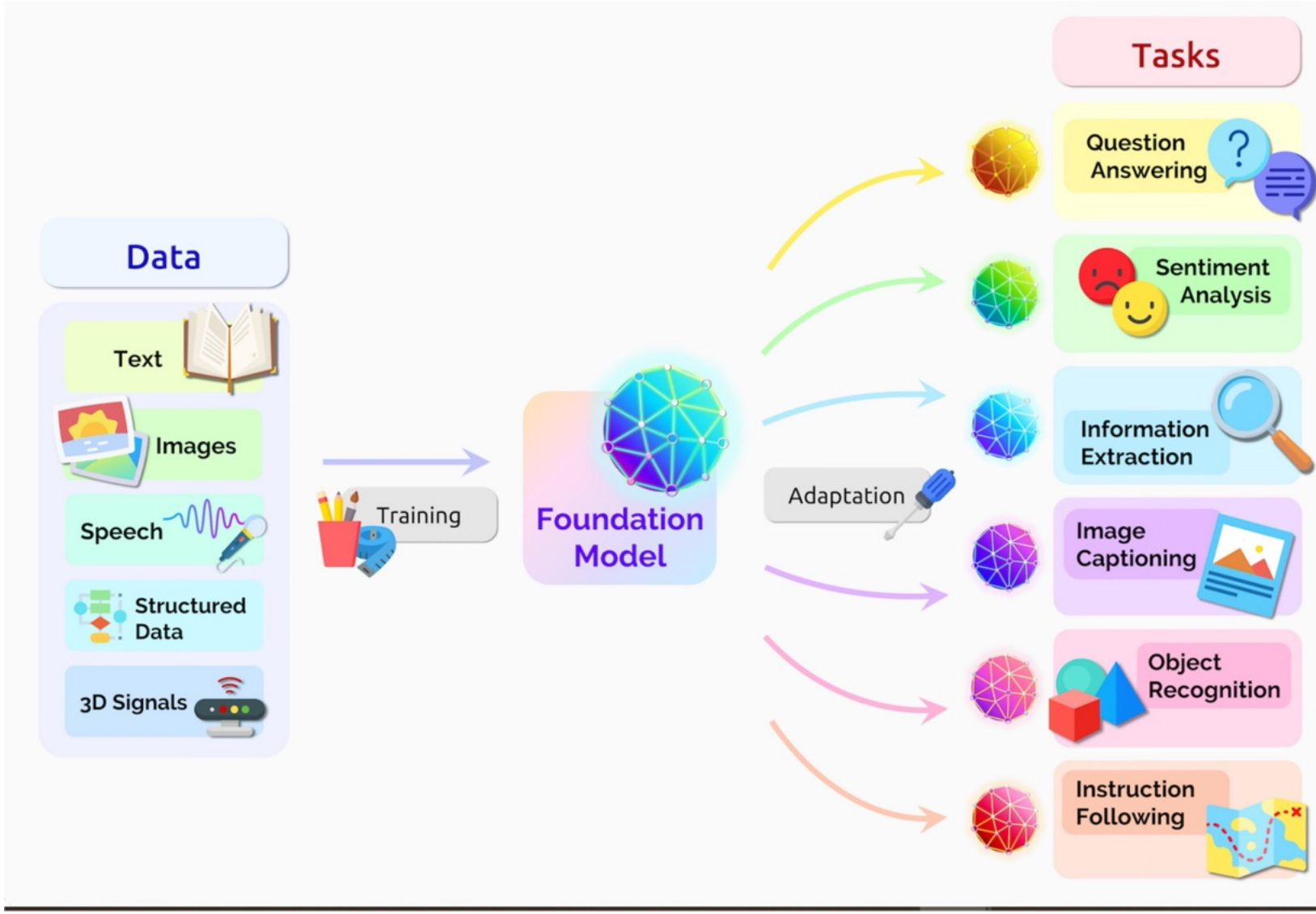
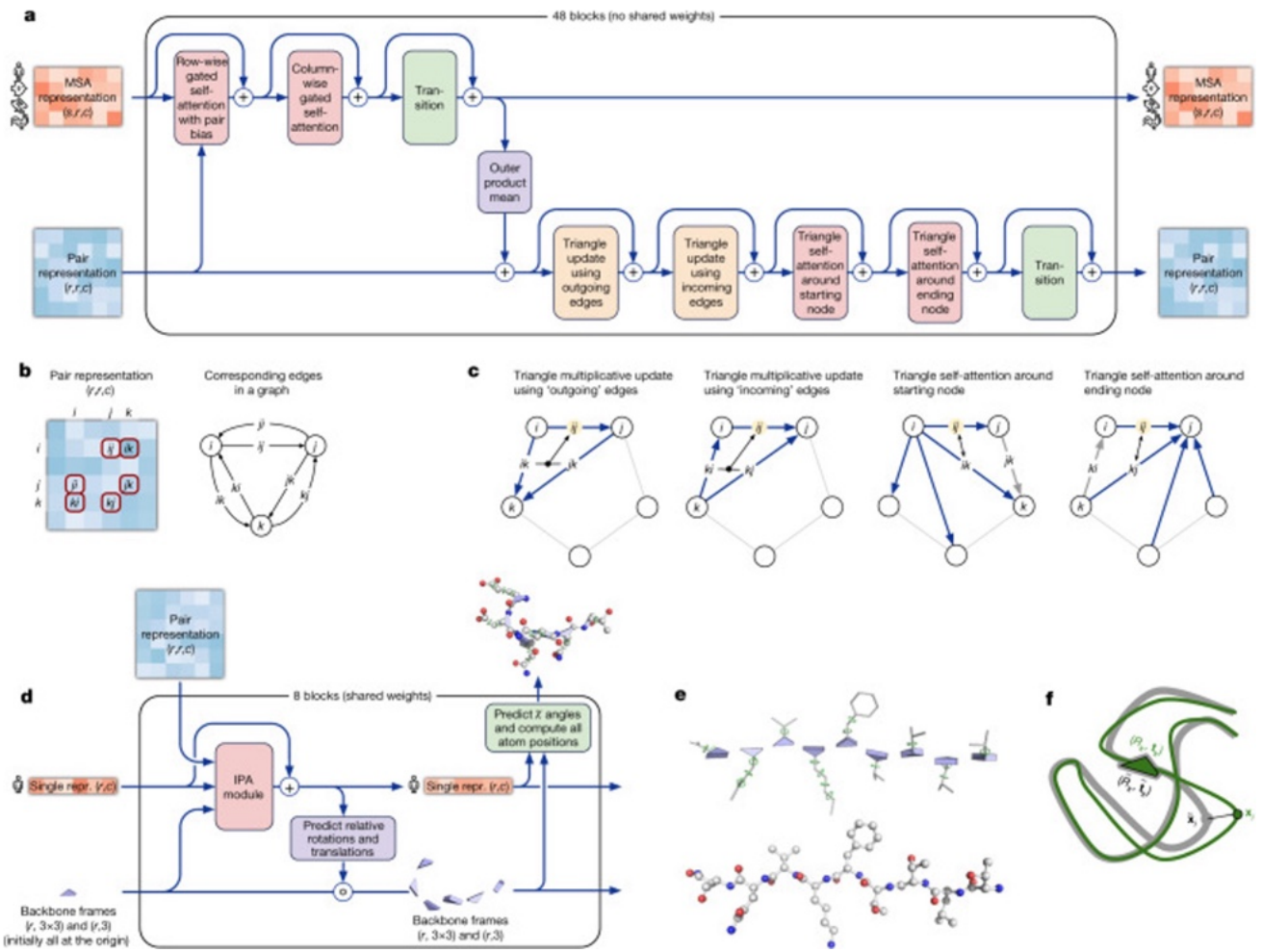


Figure 1: The Transformer - model architecture.

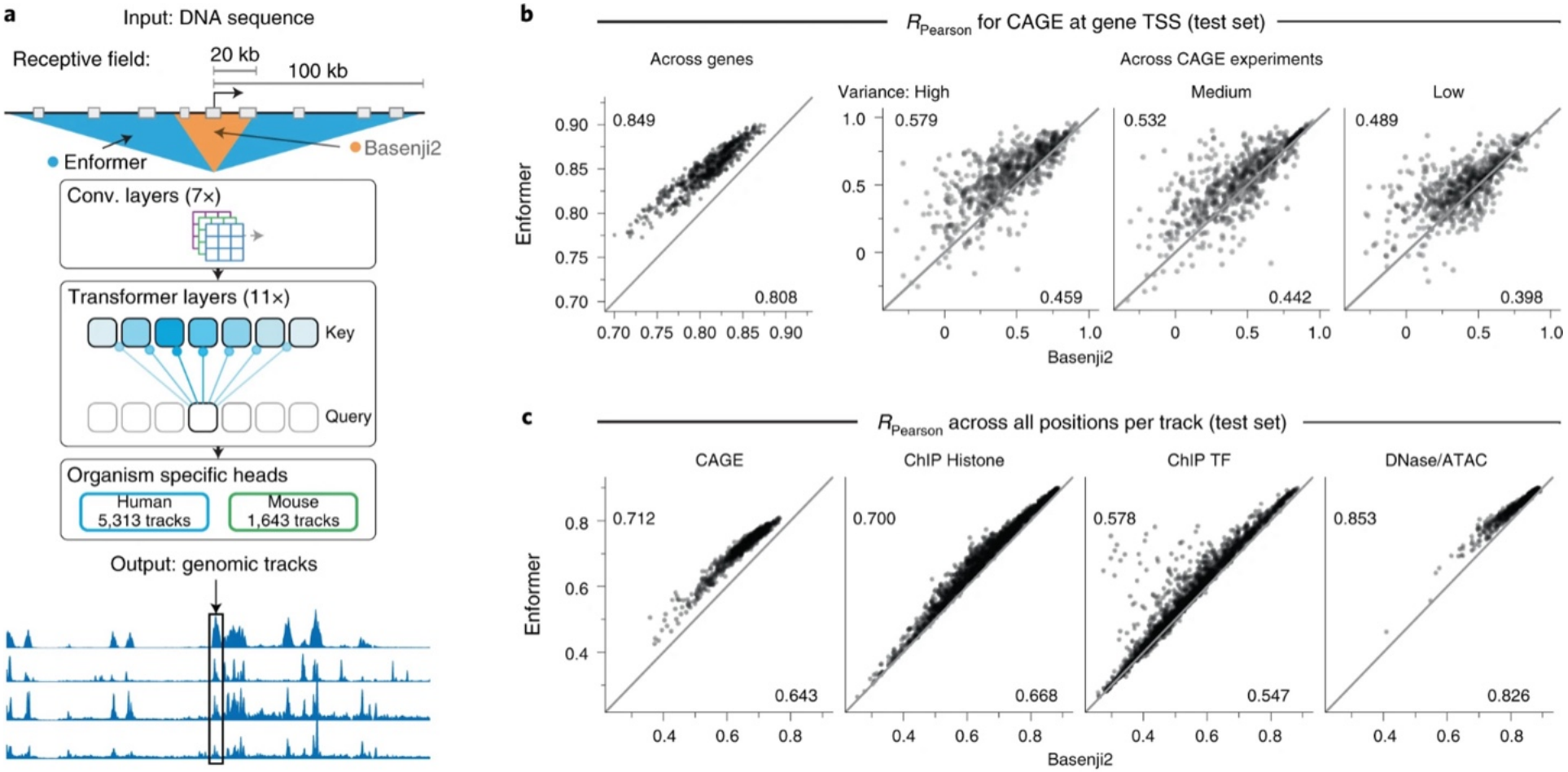
Image to text

Text to image


Data
Text
Images
Speech
Structured Data
3D Signals
Training
Foundation Model
Adaptation
Tasks
Question Answering
Sentiment Analysis
Information Extraction
Image Captioning
Object Recognition
Instruction Following



Highly accurate protein structure prediction with AlphaFold ("Evoformer")
Jumper et al. (2021) Nature. doi: 10.1038/s41586-021-03819-2



“Enformer” Effective gene expression prediction from sequence by integrating long-range interactions
Avsec et al. (2021) Nature Methods. https://doi.org/10.1038/s41592-021-01252-x

Discussion

Transformers and related architectures are transforming language processing and related tasks: attention is all you need

- Attention finds for a given term which other terms are important
- Transfomers are great for language processing by computing progressively more abstract features and concepts in a very high dimensional space

Questions:

- DNA sequences share some properties with natural language
- Natural language has a lot of semantics, and is constrained by human brain
- If we apply to bioinformatics, do the same semantics and constraints apply?
- Some parts yes (Enformer is working), some parts need more research!